

Universidade do Minho
Escola de Engenharia
Licenciatura em Engenharia Informática
Aprendizagem e Decisão Inteligentes
Ano Letivo 2025/2026

Relatório de Resultados

Análise de Dados e Modelação com KNIME

Spotify Tracks & Consumo Energético

Grupo 11

Marco Sèvegrand A106807
Francisco Martins A106902
Nuno Rebelo A106825
Marco Ferreira A106933

15 de maio de 2026

Conteúdo

1	Introdução	1
	Parte I — Tarefa 1: Dataset de Grupo (Spotify Tracks)	2
2	Domínio e objetivos	2
2.1	Domínio analisado	2
2.2	Objetivos	2
3	Metodologia CRISP-DM	3
3.1	Compreensão do negócio	3
3.2	Compreensão dos dados	3
3.3	Preparação dos dados	5
3.3.1	Remoção de duplicados	5
3.3.2	Sanitização e engenharia de atributos	5
3.3.3	Imputação de valores em falta	6
3.3.4	Tratamento de <i>outliers</i>	7
3.3.5	Seleção de variáveis e separação de ramos	7
3.4	Modelação	7
3.4.1	<i>Clustering</i> de perfis acústicos	7
3.4.2	Classificação de género musical	9
3.4.3	Regressão da popularidade	12
3.5	Avaliação	13
4	Resultados e análise crítica	13
4.1	Leitura crítica do <i>clustering</i>	13
4.2	Leitura crítica da classificação de género	14
4.3	Relação entre <i>clustering</i> e classificação	14
4.4	Leitura crítica da regressão	15
4.5	Limitações	16
	Parte II — Tarefa 2: Dataset Atribuído (Consumo Energético)	20
5	Definição da metodologia	20
5.1	Domínio e dataset	20
5.2	Visão geral do workflow desenvolvido no KNIME	20
6	Exploração	20
7	Transformação	22
8	Modelação e Avaliação	23
8.1	Decision Tree	23
8.2	Random Forest	24
8.3	Gradient Boosted Trees	25
8.4	Multilayer Perceptron	26
8.5	Keras	28

9	Conclusão	31
9.1	Síntese dos resultados do dataset atribuído	31
9.2	Síntese global do trabalho	32

1 Introdução

O presente relatório documenta o trabalho desenvolvido no âmbito da unidade curricular de Aprendizagem e Decisão Inteligentes, correspondente à componente prática de grupo. O objetivo central consistiu em conceber, implementar e avaliar modelos de aprendizagem automática sobre dois conjuntos de dados distintos, recorrendo à plataforma KNIME como ambiente de desenvolvimento e experimentação.

O trabalho estrutura-se em duas tarefas independentes, conforme definido no enunciado. A **Tarefa 1** incide sobre um **dataset de grupo** — um conjunto de dados sobre faixas musicais do Spotify, gerado sinteticamente para simular um cenário realista de preparação de dados. Nesta tarefa, a metodologia adotada foi o **CRISP-DM**, aplicando-se técnicas de *clustering* para identificação de perfis acústicos, modelos de classificação para previsão do género musical e modelos de regressão para previsão da popularidade das faixas. A **Tarefa 2** é dedicada a um **dataset atribuído** — um conjunto de dados sobre consumo energético e variáveis meteorológicas, distribuído pela equipa docente através do Blackboard. Nesta tarefa, seguiu-se a metodologia **SEMMA**, desenvolvendo-se modelos de classificação com base nas relações entre variáveis climáticas e os padrões de consumo registados.

Cada tarefa é descrita em parte própria do relatório, com a respetiva fundamentação metodológica, descrição das fases de análise e modelação, e discussão crítica dos resultados.

Parte I — Tarefa 1: Dataset de Grupo (Spotify Tracks)

2 Domínio e objetivos

2.1 Domínio analisado

O dataset `spotify_tracks.csv` foi gerado por um *script* Python especificamente concebido para simular um conjunto de dados realista sobre faixas musicais da plataforma Spotify. O ficheiro contém **100 500 registos** e **21 colunas**, cobrindo o período temporal de **2000 a 2024** e **20 géneros** musicais distintos. Cada registo representa uma faixa, caracterizada por atributos de identificação, variáveis de contexto editorial e um conjunto de *features* acústicas extraídas da API do Spotify.

O domínio analisado é o da análise musical e do consumo digital. As *features* acústicas — como `danceability`, `energy`, `loudness` e `acousticness` — capturam propriedades sonoras objetivas, enquanto variáveis como `genre`, `release_year` e `explicit` fornecem contexto editorial. O target principal é `popularity`, uma pontuação numérica entre 0 e 100.

O gerador não produziu um *dataset* limpo; pelo contrário, introduziu deliberadamente problemas de qualidade comuns em cenários reais de ciência de dados. Foram injetados valores em falta, *outliers* numéricos em várias colunas e 500 linhas duplicadas. Existe ainda um ficheiro auxiliar, `spotify_tracks_errors_log.csv`, que regista a localização exata de cada erro introduzido e serviu como referência para validação das etapas de preparação.

Tabela 1: Estrutura do dataset Spotify Tracks

Grupo	Colunas
Identificação	<code>track_id</code> , <code>track_name</code> , <code>artist_name</code> , <code>album_name</code>
Contexto	<code>release_year</code> , <code>genre</code> , <code>explicit</code> , <code>key</code> , <code>mode</code> , <code>time_signature</code>
Target	<code>popularity</code>
Áudio	<code>danceability</code> , <code>energy</code> , <code>loudness</code> , <code>speechiness</code> , <code>acousticness</code> , <code>instrumentalness</code> , <code>liveness</code> , <code>valence</code> , <code>tempo</code> , <code>duration_ms</code>

2.2 Objetivos

Os objetivos definidos para esta tarefa foram:

- garantir uma preparação robusta dos dados, com tratamento de duplicados, valores em falta, *outliers* e criação de atributos derivados;
- identificar perfis acústicos através de *k-means* e interpretar a composição dos *clusters* obtidos;
- treinar modelos de classificação para prever o género musical (`genre`, 20 classes) a partir das *features* acústicas, relacionando esta abordagem supervisionada com a segmentação

não supervisionada do *clustering*;

- treinar modelos de regressão para prever **popularity** a partir das variáveis acústicas, de contexto e do género musical (codificado em *one-hot*);
- produzir uma análise crítica dos resultados, com foco na relação entre *clustering* e classificação, na contribuição do género para a regressão e no desempenho preditivo cruzado das três vertentes.

3 Metodologia CRISP-DM

A metodologia CRISP-DM (*Cross-Industry Standard Process for Data Mining*) organiza o processo de análise de dados em seis fases iterativas: compreensão do negócio, compreensão dos dados, preparação dos dados, modelação, avaliação e *deployment*. Esta metodologia orientou a estrutura do *workflow* KNIME e a organização desta secção do relatório.

3.1 Compreensão do negócio

O objetivo analítico é triplo: caracterizar faixas musicais através de perfis acústicos (segmentação), prever o género musical a partir dessas características (classificação) e prever a popularidade com base nas propriedades sonoras e de contexto (regressão). Na prática, estes objetivos traduzem-se em casos de uso como segmentação de catálogo, recomendação musical e análise do grau em que os atributos de áudio explicam o sucesso comercial de uma faixa.

Do ponto de vista do negócio, importa perceber se as *features* acústicas fornecidas pela API do Spotify são suficientemente informativas para discriminar géneros musicais e prever popularidade. Esta pergunta tem valor prático imediato: uma plataforma de *streaming* pode usar estes modelos para etiquetagem automática de catálogo, curadoria de playlists e análise preditiva de desempenho.

3.2 Compreensão dos dados

O bloco de *Data Understanding* do *workflow* foi implementado com 31 nós KNIME, organizados para cobrir todas as dimensões relevantes da qualidade dos dados. A leitura inicial, através do nó **CSV Reader (#2)**, confirmou a importação de 100 500 linhas e 21 colunas, com as *features* de áudio corretamente interpretadas como numéricas.

O nó **Statistics (#3)** foi configurado para 12 colunas numéricas — **release_year**, **popularity**, **duration_ms**, **danceability**, **energy**, **loudness**, **speechiness**, **acousticness**, **instrumentalness**, **liveness**, **valence** e **tempo** — e revelou, de forma quantitativa, os problemas centrais do dataset. As *features* acústicas apresentam cerca de 3 000 valores em falta por coluna (por exemplo: 3 013 em **danceability**, 3 021 em **speechiness** e 3 026 em **tempo**). Em paralelo, várias colunas limitadas teoricamente ao intervalo $[0, 1]$ exibem violações claras de domínio, como **danceability** com mínimo -0.299 e máximo 1.497 , **energy** com mínimo -0.298 e máximo 1.498 , e **valence** com mínimo -0.300 e máximo 1.497 . Detetaram-se ainda extremos artificiais em **loudness** (até -89.95 dB) e **tempo** (até 419.95 BPM), consistentes com a injeção deliberada de *outliers* prevista no gerador.

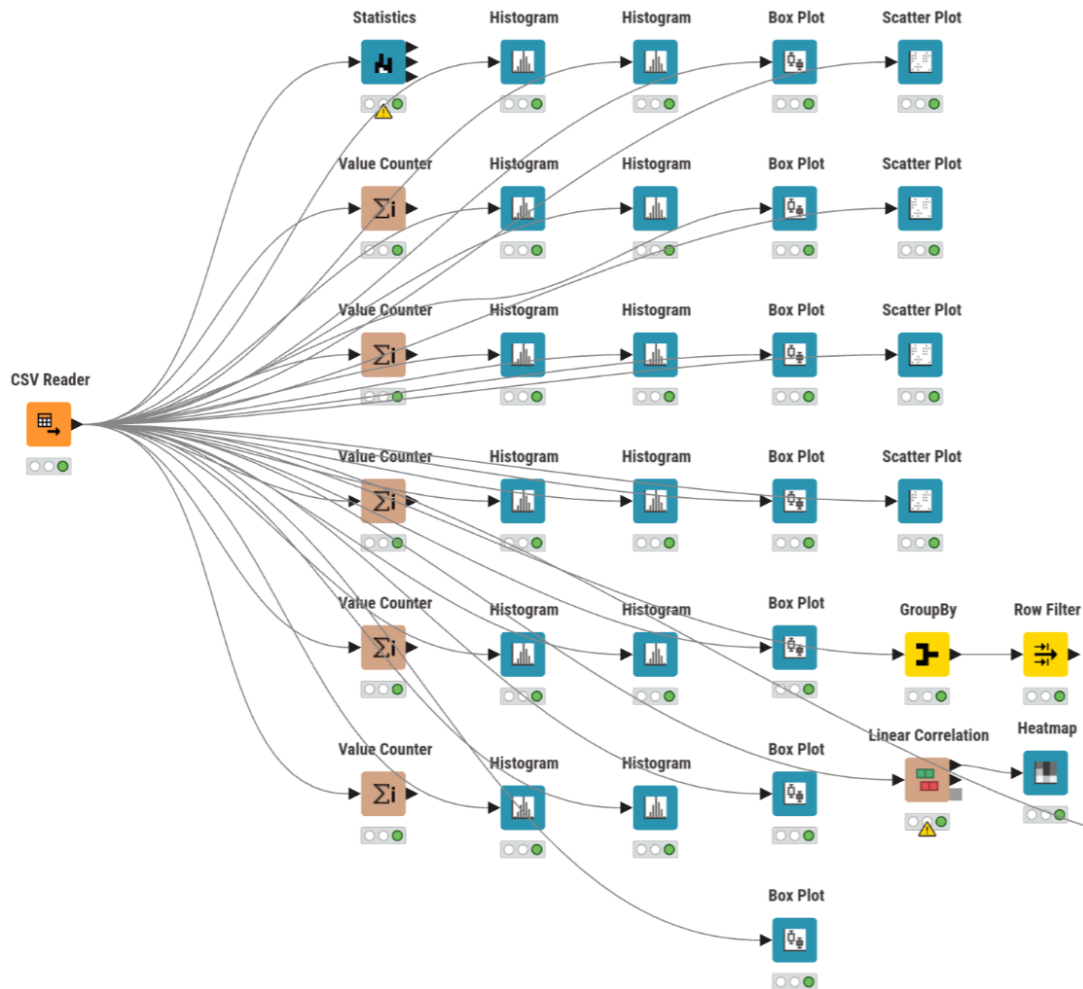


Figura 1: Workflow da fase de Compreensão dos Dados no KNIME

A componente categórica foi examinada com cinco nós **Value Counter**, cobrindo **genre** (20 géneros confirmados), **explicit**, **mode**, **key** e **time_signature**. A exploração univariada assentou em 12 histogramas e 7 *boxplots*, que documentaram as distribuições das variáveis contínuas.

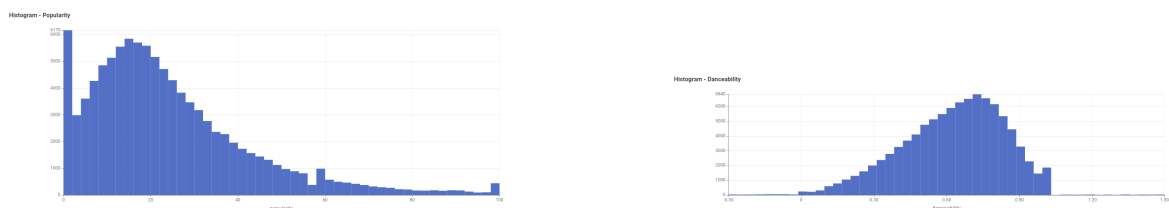


Figura 2: Distribuições de Popularity e Danceability (exemplos de histogramas exploratórios)

As relações bivariadas foram analisadas com 4 gráficos de dispersão — pares **energy-loudness**, **energy-acousticness**, **danceability-valence** e **release_year-popularity** — e uma matriz de correlação linear sobre 15 colunas, visualizada em **Heatmap** (#240) (Figura 3).

A deteção de duplicados foi realizada através do par **GroupBy** (#40) + **Row Filter** (#39), que isolou 500 registos repetidos, confirmando a injeção de duplicados prevista pelo gerador.

Em síntese, esta fase demonstrou que o *dataset* continha, de forma inequívoca, os quatro tipos de problemas que a preparação subsequente teria de resolver: valores em falta, violações

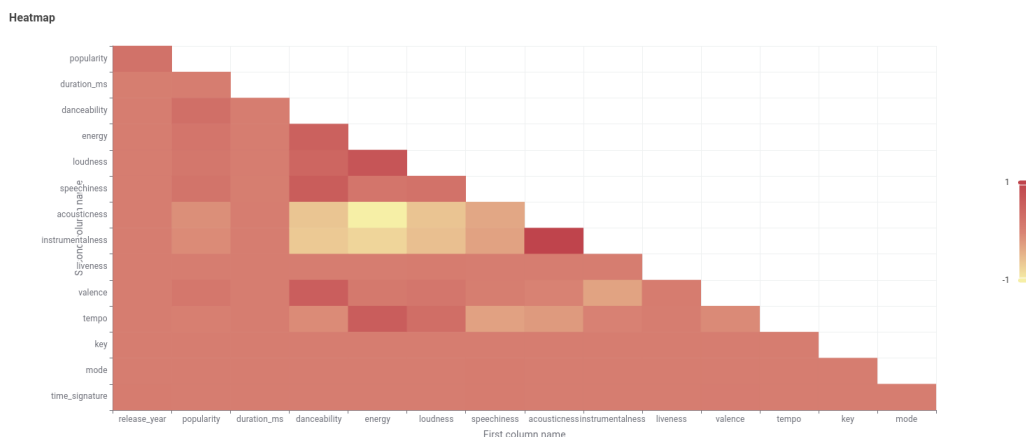


Figura 3: Matriz de correlação linear sobre as principais *features* numéricas do dataset

de domínio, *outliers* e duplicados. A Tabela 2 resume a evidência recolhida.

Tabela 2: Problemas de qualidade evidenciados na fase de compreensão dos dados

Problema	Evidência no workflow	Consequência
Valores em falta	Cerca de 3 000 missings por <i>feature</i> acústica; p.ex. 3 013 em <i>danceability</i> , 3 021 em <i>speechiness</i> , 3 026 em <i>tempo</i>	Imputação em Missing Value
Valores fora de domínio	<i>danceability</i> , <i>energy</i> e <i>valence</i> com mínimos negativos e máximos acima de 1; <i>popularity</i> validada em [0, 100]	Sanitização em Expression
Extremos artificiais	<i>tempo</i> até 419.95, <i>loudness</i> até -89.95 dB, <i>duration_ms</i> com extremos fora do perfil central	Tratamento em Numeric Outliers
Duplicados	GroupBy e Row Filter isolam exatamente 500 casos	Remoção em Duplicate Row Filter

3.3 Preparação dos dados

A preparação dos dados foi implementada como um pipeline sequencial com cinco etapas principais, executadas antes da separação entre os ramos de *clustering*, classificação e regressão.

3.3.1 Remoção de duplicados

A primeira operação do pipeline é o nó **Duplicate Row Filter** (#41), que compara todas as colunas exceto *track_id*, remove as linhas repetidas e conserva a primeira ocorrência. O resultado confirma a passagem de 100 500 para 100 000 linhas, eliminando os 500 duplicados.

3.3.2 Sanitização e engenharia de atributos

O nó **Expression** (#42) concentra num único ponto oito transformações que, num *workflow* menos compacto, estariam dispersas por múltiplos nós. As operações realizadas foram:

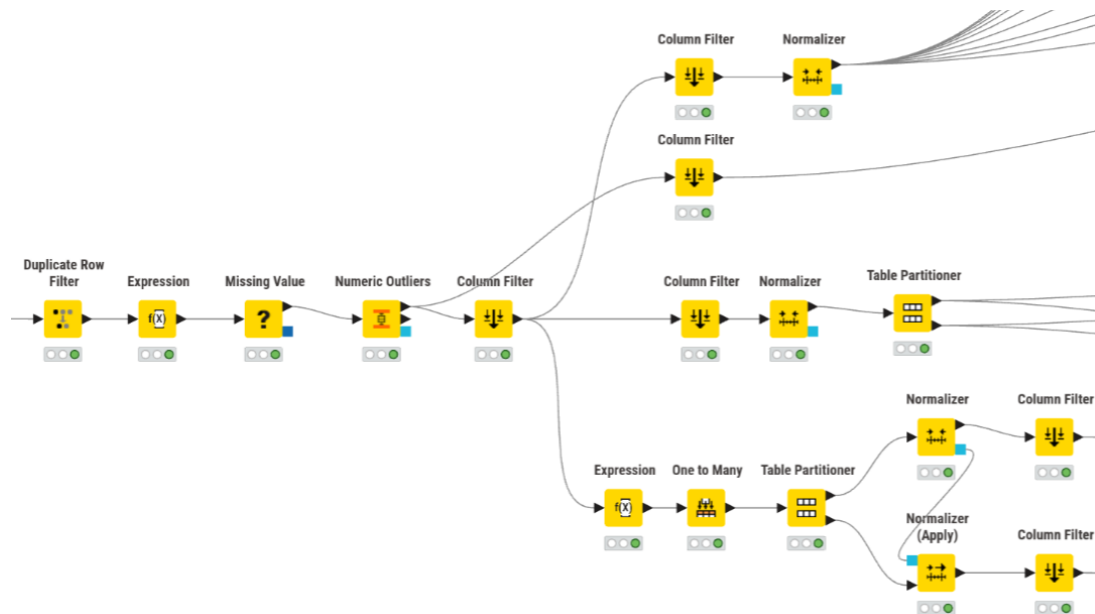


Figura 4: Workflow da fase de Preparação dos Dados no KNIME

- conversão de valores fora do domínio $[0, 100]$ de `popularity` para *missing*;
- conversão de violações do intervalo $[0, 1]$ nas *features* de áudio (`danceability`, `energy`, `speechiness`, `acousticness`, `instrumentalness`, `liveness` e `valence`) para *missing*;
- conversão de valores inválidos de `key`, `mode` e `time_signature` para *missing*;
- codificação binária da variável `explicit` (`True` $\rightarrow$ 1, `False` $\rightarrow$ 0);
- criação do atributo derivado `duration_min` ($= \text{duration_ms} / 60\,000$), mais interpretável do que milissegundos;
- criação da variável auxiliar `popularity_class` (Baixa/Media/Alta) para estratificação da partição treino/teste.

Esta etapa é metodologicamente relevante porque, em vez de eliminar precocemente as linhas problemáticas, opta por converter violações de domínio em valores em falta, delegando a decisão para o nó seguinte.

3.3.3 Imputação de valores em falta

O nó Missing Value (#43) recebe tanto os valores em falta originais do CSV como os gerados pelo Expression. A estratégia de imputação adotada não é uniforme, refletindo uma escolha ponderada por coluna:

- **Média:** `danceability`, `valence`.
- **Mediana:** `energy`, `speechiness`, `acousticness`, `instrumentalness`, `liveness`, `loudness`, `tempo`, `duration_ms`.
- **Remover linha:** `popularity` (target), caso a sanitização anterior o converta em *missing*.

O resultado executado confirma a passagem de 100 000 para **99 353 registos** — os 647 registos eliminados correspondem a linhas em que o target **popularity** ficou inválido após sanitização.

3.3.4 Tratamento de *outliers*

Os *outliers* contínuos são tratados no nó Numeric Outliers (#45), que intervém sobre três colunas — **duration_ms**, **loudness** e **tempo** — utilizando o critério IQR com fator 3.0 e substituindo os valores extremos pelo limite admissível mais próximo. Esta abordagem preserva as linhas afetadas, evitando perda adicional de dados.

3.3.5 Seleção de variáveis e separação de ramos

O nó Column Filter (#47) reduz o *dataset* à base comum efetivamente usada pelos ramos de modelação, removendo identificadores (**track_id**, **track_name**, **artist_name**, **album_name**), a coluna redundante **duration_ms** (substituída por **duration_min**) e os atributos **key**, **mode** e **time_signature**. A coluna **genre** é preservada. O resultado executado do nó apresenta **99 353 linhas e 15 colunas**, a partir das quais se abrem três ramos:

- **Ramo de *clustering***: reduzido a 10 *features* acústicas normalizadas (Min-Max para $[0, 1]$) — sem género, que é reintroduzido apenas no pós-processamento para interpretação semântica;
- **Ramo de classificação**: o Column Filter (#220) isola 11 colunas — as 10 *features* acústicas mais o target **genre** — e a Table Partitioner (#234) cria uma partição 80/20, estratificada por género e com *seed*=1, partilhada entre a MLP e a *Random Forest*;
- **Ramo de regressão**: o nó One to Many (#221) codifica **genre** em 20 colunas binárias (*one-hot encoding*), expandindo a tabela para 34 colunas totais, e a partição é feita a 80/20 estratificada por **popularity_class**.

3.4 Modelação

3.4.1 *Clustering* de perfis acústicos

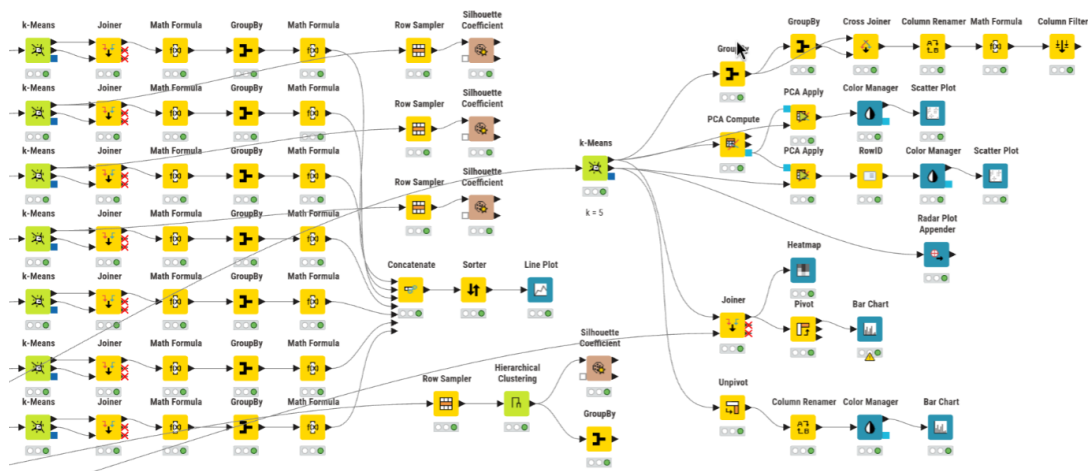


Figura 5: Workflow da fase de *Clustering* no KNIME

Método do cotovelo. A seleção do número de *clusters* baseou-se no método do cotovelo. Foram executadas sete experiências de *k-means* para valores de $k \in \{3, 4, 5, 6, 7, 8, 9\}$, com inicialização aleatória, *seed*=1 e 99 iterações máximas. Em cada ramo candidato, a soma das distâncias quadradas intra-*cluster* (WCSS) foi calculada manualmente: o nó **Joiner** associa cada ponto ao centróide do respetivo *cluster*, o nó **Math Formula** calcula a distância euclidiana quadrada às 10 coordenadas do centróide, e o nó **GroupBy** agrega os valores. As sete linhas de WCSS foram concatenadas e representadas graficamente (Figura 6).

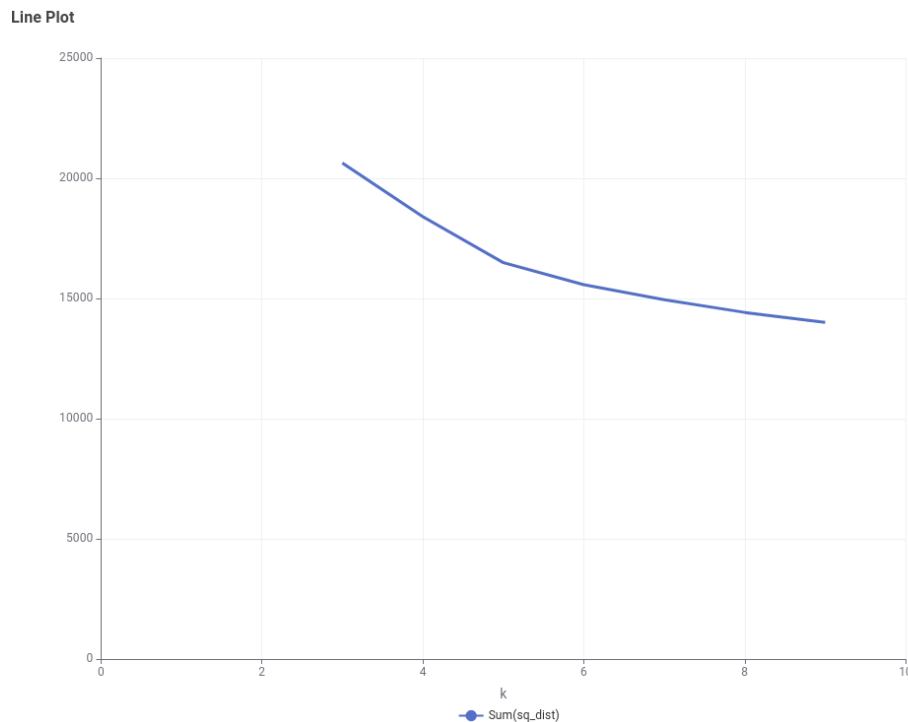


Figura 6: Soma das distâncias quadradas intra-*cluster* (WCSS) para $k \in \{3, \dots, 9\}$ — método do cotovelo

A curva apresenta uma inflexão visível entre $k = 4$ e $k = 6$, com ganhos decrescentes para valores superiores, indicando que a zona de equilíbrio entre compacidade e número de grupos se situa em torno de $k = 5$.

Modelo final. O modelo de *clustering* adotado fixou $k = 5$, coerente com a leitura da curva do cotovelo. O nó **k-Means (#141)** produz diretamente a coluna **Cluster**, dispensando um *assigner* separado. Os centróides foram transformados para formato longo (**Unpivot**) e visualizados num gráfico de barras agrupado (Figura 7), que permite comparar o perfil acústico médio de cada *cluster* nas 10 *features* normalizadas.

Interpretação. A dimensão dos *clusters* foi calculada através de **GroupBy** e **Cross Joiner**, obtendo-se a percentagem relativa de cada grupo. Para interpretação semântica, a coluna **genre** foi reintroduzida via **Joiner** por **RowID**, permitindo construir uma matriz **Cluster** $\times$ **Genre** com **Pivot** e um gráfico de barras empilhadas (Figura 8).

Esta visualização confirma que os agrupamentos capturam afinidades estilísticas genuínas: o *cluster_0* apresenta o perfil mais acústico e instrumental (**acousticness** = 0.708; **instrumentalness** = 0.751), concentrando géneros como *classical* e *ambient*; o *cluster_1* concentra as faixas mais energéticas e rápidas (**energy** = 0.843; **tempo** = 0.614); e o *cluster_2* distingue-se



Figura 7: Perfil acústico médio de cada *cluster* ($k = 5$) nas 10 *features* normalizadas

pelo nível muito elevado de **danceability** (0.806) e **speechiness** (0.581), aproximando-se de géneros de matriz urbana como *hip-hop*.

Validação. A qualidade do agrupamento foi avaliada com o coeficiente de *silhouette*. O método hierárquico (*Euclidean, linkage complete*) operou sobre uma amostra de **5 000 linhas** e produziu um valor médio global de **0.0883**. Para os modelos *k-means*, os valores médios observados foram **0.1802** para $k = 4$, **0.1805** para $k = 5$ e **0.1619** para $k = 6$, calculados sobre amostras estratificadas por *cluster*. A escolha final de $k = 5$ apoia-se na conjugação entre o método do cotovelo e a maior interpretabilidade semântica dos cinco perfis acústicos resultantes.

A projeção PCA bidimensional dos pontos (Figura 9) complementa a análise, confirmando a estrutura de sobreposição sugerida pelos valores de *silhouette*.

3.4.2 Classificação de género musical

A novidade face ao ramo de *clustering* é a inclusão de um ramo de classificação supervisionada que utiliza as mesmas 10 *features* acústicas, mas com o objetivo de prever o género musical (20 classes). Este desenho permite contrastar diretamente a aprendizagem não supervisionada (agrupamento por semelhança acústica, sem conhecimento dos rótulos) com a aprendizagem supervisionada (classificação informada pelo género real), medindo até que ponto os atributos de áudio contêm sinal suficiente para discriminar estilos musicais.

Configuração. O ramo de classificação parte da base comum pós-preparação. O nó **Column Filter** (#220) isola 11 colunas: as 10 *features* acústicas e o target **genre**. A normalização Min-Max ([0, 1]) é aplicada através do nó **Normalizer** (#218), garantindo que todas as variáveis contribuam em escala comparável.

Foram treinados dois modelos supervisionados:

- **RProp MLP** (#226): rede neuronal *feedforward* com **duas camadas escondidas** de 30 neurónios, treinada com o algoritmo *RProp* durante 400 iterações (*seed*=1). Usa a partição

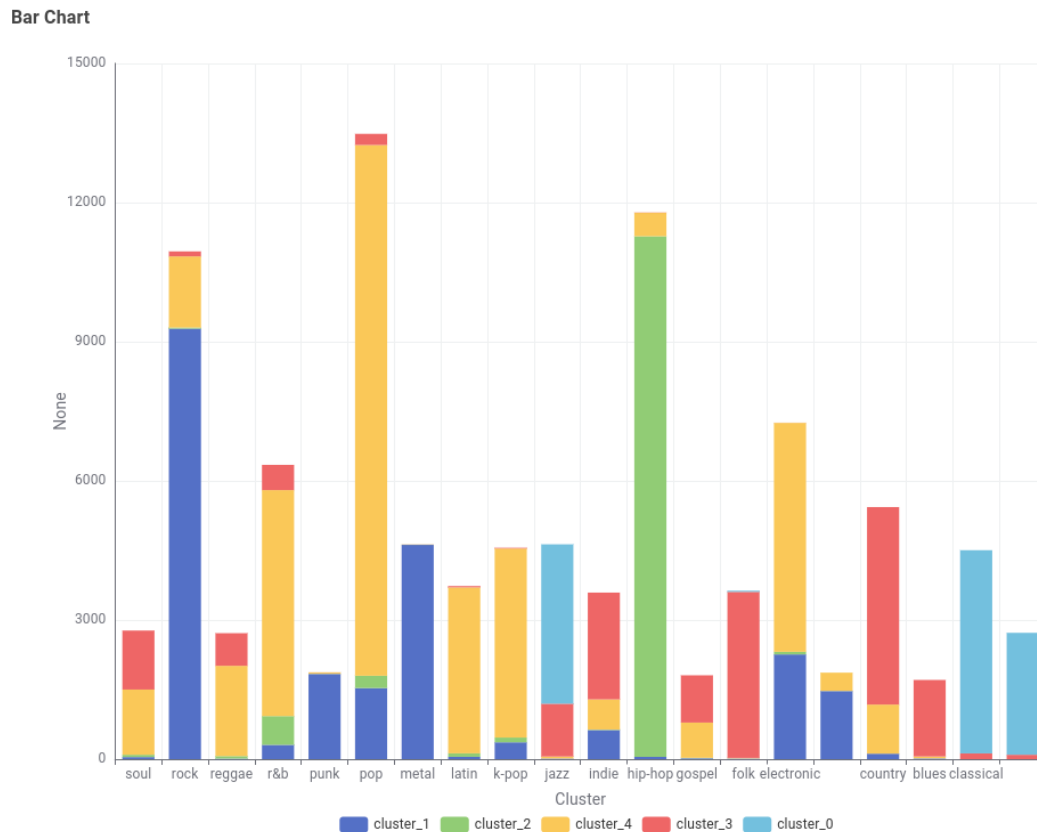


Figura 8: Distribuição dos géneros musicais por *cluster* (barras empilhadas)

80/20 estratificada por género (80 000 exemplos de treino e 20 000 de teste).

- **Random Forest (#235):** *ensemble* de **100 árvores** de decisão, critério Gini, profundidade máxima 20, amostragem de colunas em modo $\sqrt{\cdot}$ (*SquareRoot*), com *seed*=1. Usa a mesma partição 80/20 estratificada por género.

O uso de uma partição partilhada entre os dois classificadores torna a comparação direta metodologicamente mais limpa: MLP e *Random Forest* são avaliadas sobre exatamente o mesmo conjunto de teste. Para a *Random Forest*, foi ainda realizado um estudo de sensibilidade sobre configurações diversas, incluindo diferentes profundidades, critérios de divisão e número de árvores.

Resultados. A avaliação foi realizada através do nó **Scorer**, que calcula a exatidão (*accuracy*), o erro, o número de classificações corretas e incorretas, e o coeficiente Kappa de Cohen. A Tabela 3 sintetiza os resultados obtidos.

Tabela 3: Métricas de classificação — Previsão de género musical (20 classes)

Modelo	Exatidão	Erro	Corretas	Kappa
RProp MLP (80/20, 400 iter, 2 hidden layers 30)	0.6815	0.3185	13 630	0.6542
<i>Random Forest</i> (80/20, 100 árvores, Gini, prof. 20)	0.6838	0.3162	13 675	0.6564
<i>Random Forest</i> melhor config. (gini_20_1_500)	0.6844	0.3156	13 688	0.6571

Análise por classe. A matriz de confusão visualizada no *Heatmap* (Figura 11) permite

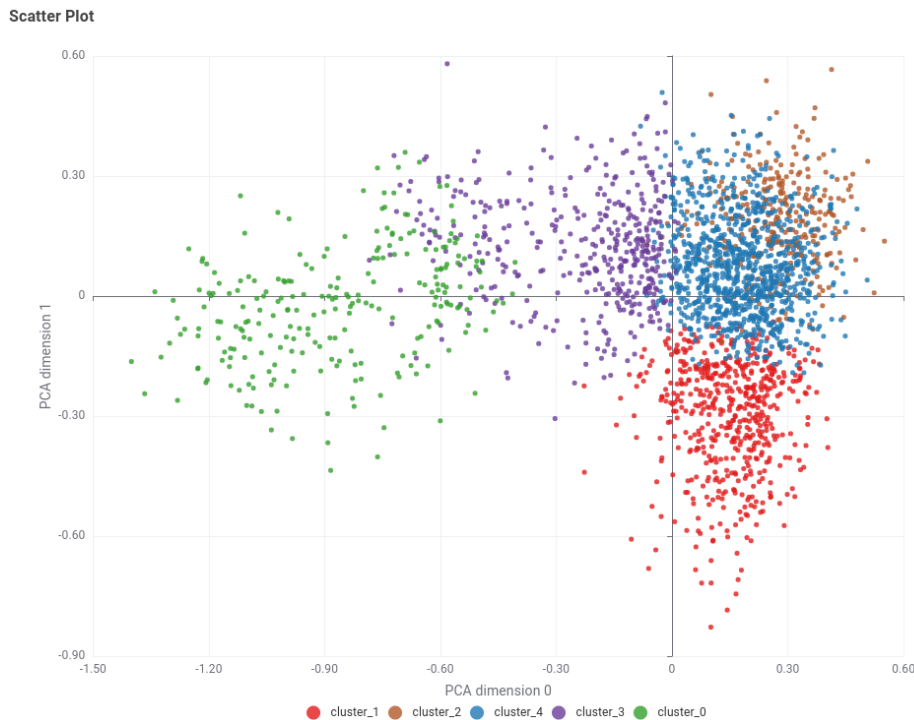


Figura 9: Projeção PCA dos pontos e centróides (duas primeiras componentes), coloridos por *cluster*

inspecionar a distribuição dos erros por classe, revelando que alguns géneros são classificados quase perfeitamente enquanto outros apresentam erros frequentes.

A *Random Forest* do ramo principal distingue quase perfeitamente géneros com assinatura acústica mais estável, como *jazz* ($\text{recall} \approx 0.990$), *classical* (≈ 0.973), *hip-hop* (≈ 0.951) e *ambient* (≈ 0.951). Em contrapartida, classes como *soul* (≈ 0.293), *k-pop* (≈ 0.422), *r&b* (≈ 0.450) e *latin* (≈ 0.456) revelam separação muito mais fraca, refletindo a maior sobreposição acústica destes géneros.

A projeção PCA das previsões da *Random Forest* (Figura 12) confirma visualmente esta sobreposição: as diferentes classes formam manchas com contornos difusos, com algumas classes mais compactas e outras dispersas.

A projeção PCA das previsões da RProp MLP confirma padrões semelhantes (Figura 13).

Estudo de sensibilidade da *Random Forest*. Foram testadas várias configurações adicionais de *Random Forest*, variando o critério de divisão (Gini vs. *Information Gain Ratio*), a profundidade máxima, o tamanho mínimo de nó e o número de árvores. Os resultados mostram que a partir das configurações intermédias o desempenho converge para um patamar estreito: 68.16% (gini_16_5_100), 68.38% (gini_20_1_100), 68.36% (gini_20_1_200) e 68.44% (gini_20_1_500). A troca do critério Gini por *Information Gain Ratio* não melhora o desempenho em configurações comparáveis (68.38% vs. 68.20%), indicando que a fronteira de decisão é relativamente robusta à variação do critério de divisão.

A Tabela 4 resume as configurações mais representativas do estudo.

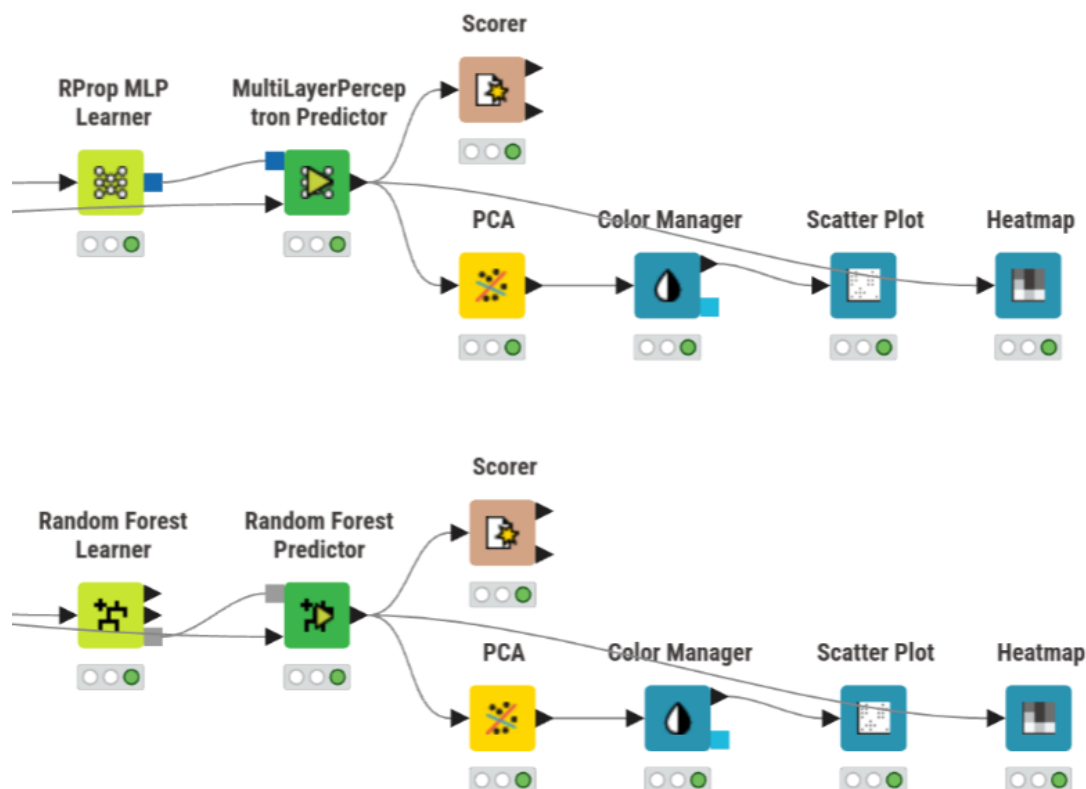


Figura 10: Workflow da fase de Classificação no KNIME

3.4.3 Regressão da popularidade

Configuração. O ramo de regressão foi expandido para incluir o género musical em *one-hot encoding*. Além dos **12 preditores** originais (`release_year`, `explicit` e as 10 *features* de áudio, incluindo `duration_min`), o nó *One to Many* (#221) codifica os 20 géneros musicais em colunas binárias, totalizando **32 preditores** úteis para os modelos. Esta expansão concretiza a hipótese de que o género musical transporta informação complementar às *features* acústicas para explicar a variabilidade da popularidade. A normalização Min-Max é aprendida no treino e aplicada ao teste, sem fuga de informação.

Foram treinados três modelos: regressão linear múltipla, árvore de regressão simples e *Random Forest*, todos com 32 preditores e target `popularity`.

Regressão linear múltipla. O modelo linear, treinado com *Linear Regression Learner* (#163), inclui constante e utiliza todos os preditores disponíveis. As previsões foram avaliadas com *Numeric Scorer* (#165). O diagnóstico visual incluiu um gráfico de dispersão de valores reais vs. previstos e um histograma dos resíduos (Figura 15).

Árvore de regressão. Para a árvore de regressão, o *workflow* contém um ramo principal com *Simple Regression Tree Learner* (#166), e foram avaliadas configurações exportadas com diferentes níveis de poda/regularização. Entre as experiências, a melhor foi a configuração 5_40_20 (profundidade máxima 5, mínimo de registos por nó 40, tamanho mínimo de nó filho 20). Os diagnósticos encontram-se na Figura 16.

A Tabela 5 apresenta o estudo de configurações da árvore de regressão.

Random Forest. O modelo de *Random Forest*, configurado no nó *Random Forest Learner*

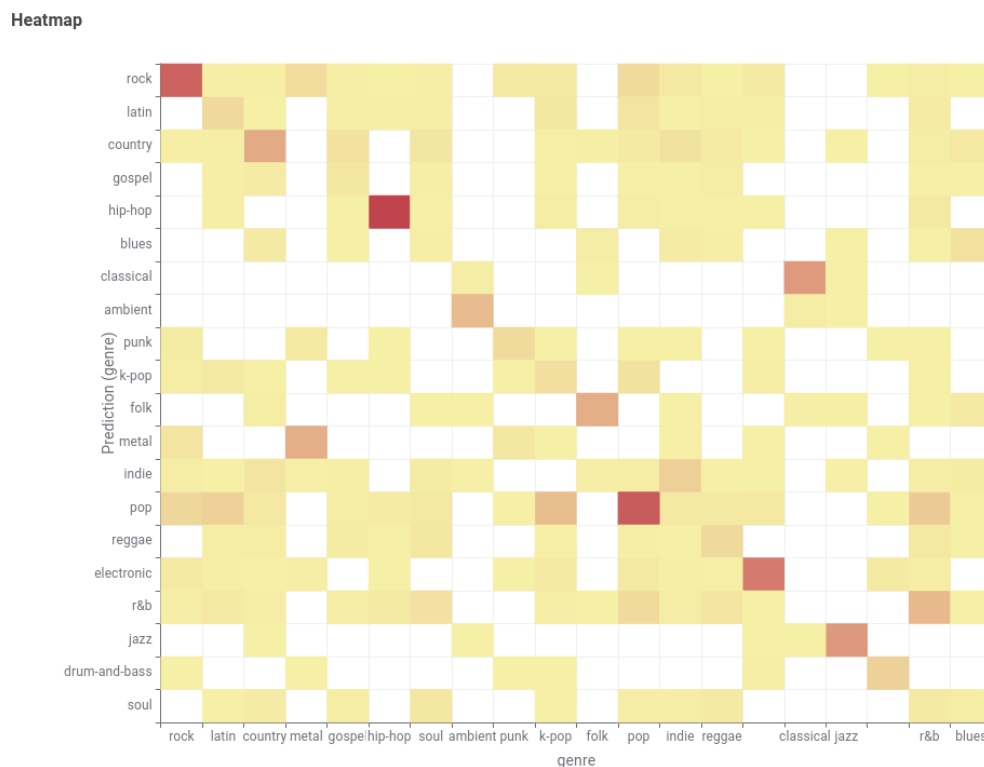


Figura 11: Matriz de confusão — classificação de gênero via *Random Forest* (20 classes)

(Regression) (#172), utiliza 100 árvores, *seed*=1, *bootstrap* ativo e amostragem de colunas em modo $\sqrt{\cdot}$ (*SquareRoot*). Os diagnósticos encontram-se na Figura 17.

Os resultados adicionais da *Random Forest* de regressão apresentam-se na Tabela 6, comparando três configurações.

3.5 Avaliação

A avaliação quantitativa da regressão baseou-se em R^2 , MAE e RMSE, calculados via **Numeric Scorer**. A Tabela 7 resume os resultados comparativos dos três modelos com a melhor configuração de cada.

4 Resultados e análise crítica

4.1 Leitura crítica do *clustering*

Os resultados de *clustering* mostram que existe estrutura nos dados musicais, mas a separação entre grupos não é nítida. Um coeficiente de *silhouette* de 0.0883 no método hierárquico constitui um sinal de sobreposição relevante entre *clusters*, o que é consistente com a natureza do domínio: diferentes faixas podem partilhar níveis semelhantes de energia, valência e dançabilidade sem que isso se traduza em grupos radicalmente distintos. Em termos práticos, o *clustering* revela-se mais útil como ferramenta de segmentação exploratória do que como mecanismo de rotulagem rígida.

A análise dos centróides por *feature* permite identificar perfis sonoros diferenciados, e a composição dos *clusters* por gênero musical confirma que os agrupamentos capturam afinidades estilísticas genuínas, ainda que com sobreposição. Os valores médios de *silhouette* para *k-means*

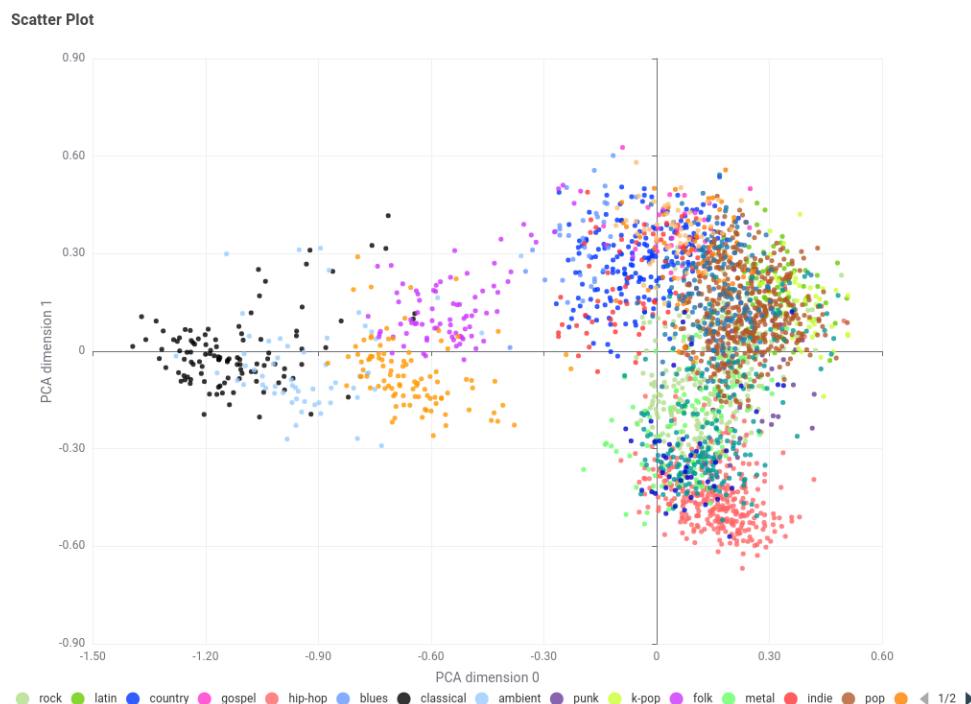


Figura 12: Projeção PCA das previsões da *Random Forest* (pontos coloridos por gênero previsto)

foram 0.1802 para $k = 4$, 0.1805 para $k = 5$ e 0.1619 para $k = 6$, mostrando que $k = 4$ e $k = 5$ são praticamente equivalentes em qualidade geométrica. A preferência por $k = 5$ resulta, assim, menos de um ganho quantitativo e mais de uma melhor legibilidade semântica dos perfis obtidos.

4.2 Leitura crítica da classificação de gênero

Os resultados de classificação demonstram que as 10 *features* acústicas contêm sinal discriminativo substancial para a previsão do gênero musical. No ramo principal do *workflow*, a *Random Forest* atinge 68.38% de exatidão e a MLP 68.15%, ambas com Kappa em torno de 0.65. Este resultado é expressivo quando comparado com a *baseline* aleatória de 5% (1/20 gêneros), mas a proximidade entre os dois modelos indica que a capacidade preditiva está menos limitada pela escolha do algoritmo e mais pela estrutura disponível nas *features*. Nas experiências suplementares exportadas, a melhor *Random Forest* sobe apenas até **68.44%**, reforçando a ideia de saturação.

A leitura por classe mostra um comportamento muito desigual: gêneros com assinatura acústica mais estável (jazz, classical, hip-hop) são classificados quase perfeitamente, enquanto gêneros mais híbridos (soul, k-pop, r&b) revelam separação muito mais fraca. Esta assimetria sugere que a dificuldade do problema reside menos na complexidade do classificador e mais na sobreposição intrínseca entre classes no espaço acústico.

4.3 Relação entre *clustering* e classificação

A coexistência dos dois ramos sobre o mesmo espaço de *features* permite uma triangulação metodológica com conclusões convergentes:

1. **Ambas as abordagens encontram sinal.** O *k-means* identifica 5 grupos acústicos não aleatórios (*silhouette* positivo e estável entre 0.1619 e 0.1805). A classificação supervisio-

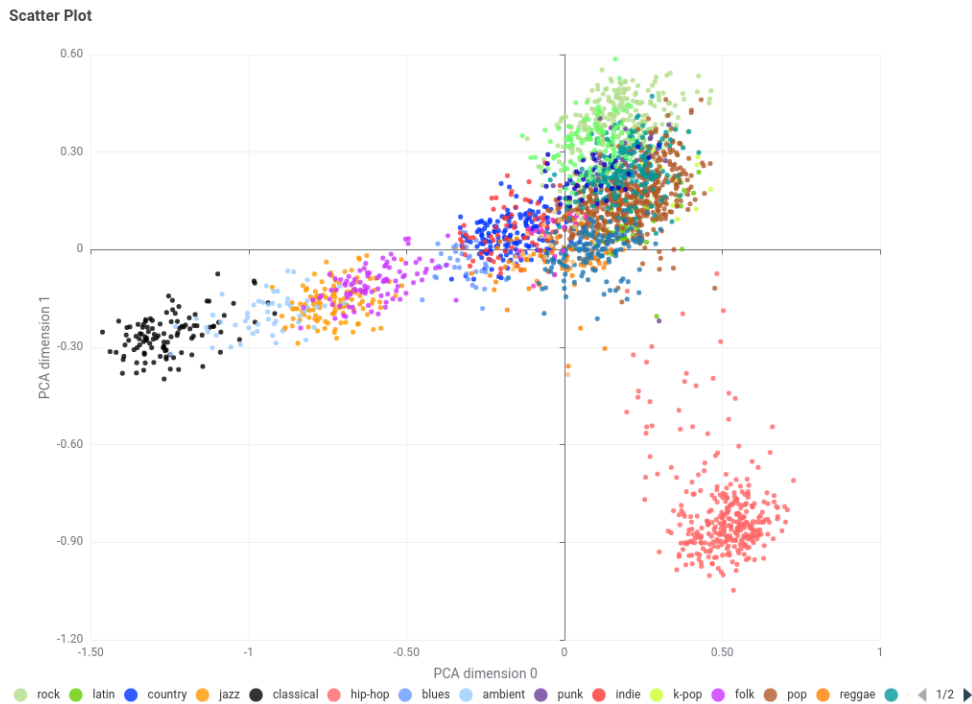


Figura 13: Projeção PCA das previsões da RProp MLP (pontos coloridos por género previsto)

nada recupera o género com 68.38% de exatidão, muito acima dos 5% do acaso. As *features* acústicas têm, de facto, poder discriminativo.

2. **A separação não é total.** A *silhouette* modesta é a contrapartida não supervisionada da exatidão de classificação de cerca de 68.4% — ambas apontam para uma sobreposição intrínseca no espaço acústico que nenhum modelo consegue eliminar.
3. **O *clustering* descobre agrupamentos que a classificação valida.** A matriz *cluster* × *género* mostra que géneros próximos no espaço acústico coabitam os mesmos *clusters*, e a matriz de confusão da classificação revela que as trocas entre géneros seguem precisamente este mesmo padrão de afinidade acústica. Esta convergência entre técnica não supervisionada e supervisionada reforça a validade de ambas.
4. **Limite fundamental da representação acústica.** O facto de mesmo as melhores configurações testadas não ultrapassarem 68.44% de exatidão estabelece um limite superior empírico para o que é possível extrair das 10 *features* de áudio. Este limite explica por que razão o *k-means* produz *clusters* com *silhouette* modesta.

4.4 Leitura crítica da regressão

A regressão da popularidade confirma a natureza estruturalmente difícil do target. Mesmo após a inclusão do género por *one-hot encoding*, todos os modelos permanecem com R^2 muito baixos: a regressão linear é a melhor das três abordagens com $R^2 = 0.0459$, seguida pela melhor árvore afinada com $R^2 = 0.0346$, e pela *Random Forest* com $R^2 = 0.0321$. Os três modelos operam num intervalo muito estreito de erro (RMSE entre 18.19 e 18.33; MAE entre 13.72 e 13.90), o que mostra que aumentar a flexibilidade do modelo não altera materialmente a qualidade preditiva.

Tabela 4: Estudo de sensibilidade da *Random Forest* para classificação de género

Configuração	Exatidão	Kappa	Nº árvores
<code>gini_6_5_100</code>	0.5968	0.5229	100
<code>gini_8_5_100</code>	0.6350	0.5632	100
<code>gini_10_5_100</code>	0.6614	0.6120	100
<code>gini_12_5_100</code>	0.6756	0.6397	100
<code>gini_14_5_100</code>	0.6797	0.6443	100
<code>gini_16_5_100</code>	0.6816	0.6480	100
<code>gini_20_5_100</code>	0.6835	0.6509	100
<code>gini_20_1_100</code>	0.6838	0.6564	100
<code>gini_20_1_200</code>	0.6836	0.6561	200
<code>gini_20_1_500</code>	0.6844	0.6571	500
<code>info_gain_ratio_20_1_100</code>	0.6820	0.6543	100

Tabela 5: Estudo de configurações da árvore de regressão (Profundidade_MinRegs_MinFilho)

Configuração	R ²	RMSE	MAE
<code>3_10_5</code>	0.0254	18.411	13.914
<code>5_2_1</code>	-0.0039	18.666	14.047
<code>5_10_5</code>	0.0320	18.355	13.852
<code>5_20_10</code>	0.0335	18.340	13.831
<code>5_40_20</code>	0.0346	18.301	13.808
<code>7_10_5</code>	0.0296	18.379	13.871

O facto de a regressão linear superar os modelos não lineares sugere que o sinal disponível é simultaneamente fraco e relativamente simples. A proximidade entre a melhor árvore simples e a *Random Forest* indica que, uma vez controlado o sobreajustamento por poda, a componente não linear do problema é reduzida.

Globalmente, a regressão é a vertente onde o contraste entre *clustering*/classificação e popularidade se torna mais visível. As mesmas *features* acústicas que conseguem segmentar perfis sonoros e classificar géneros com qualidade razoável quase não conseguem prever a popularidade. Isto implica que a popularidade depende sobretudo de fatores externos ao espaço acústico: capital simbólico do artista, marketing, exposição em playlists, momento cultural e dinâmica social de consumo.

4.5 Limitações

A principal limitação é a natureza sintética do *dataset* que, embora gerado com distribuições inspiradas em dados reais, não substitui plenamente um catálogo musical autêntico. As relações entre *features* acústicas e popularidade podem ser artificialmente atenuadas ou amplificadas pelo processo de geração, e a distribuição dos géneros (uniforme, 20 géneros com igual representação) não reflete a realidade do mercado musical.

Adicionalmente, os valores de *silhouette* variam com a amostragem (0.0883 em 5 000 registos vs. 0.1619–0.1805 em amostras estratificadas), o que sugere que a estimativa da qualidade do

Tabela 6: Estudo de configurações da *Random Forest* de regressão

Configuração	R^2	RMSE	MAE
6_5_100	0.0208	18.454	13.960
nolimit_nolimit_50	0.0311	18.363	13.899
nolimit_nolimit_100	0.0321	18.325	13.896

Tabela 7: Métricas de regressão — Spotify Tracks (com género em *one-hot*)

Modelo	R^2	RMSE	MAE
Regressão linear múltipla	0.0459	18.194	13.718
Árvore de regressão (5_40_20)	0.0346	18.301	13.808
<i>Random Forest</i> (nolimit_nolimit_100)	0.0321	18.325	13.896

agrupamento é sensível ao tamanho da amostra. As métricas de regressão podem ainda ser influenciadas pelas estratégias de imputação e de tratamento de *outliers* adotadas na preparação.

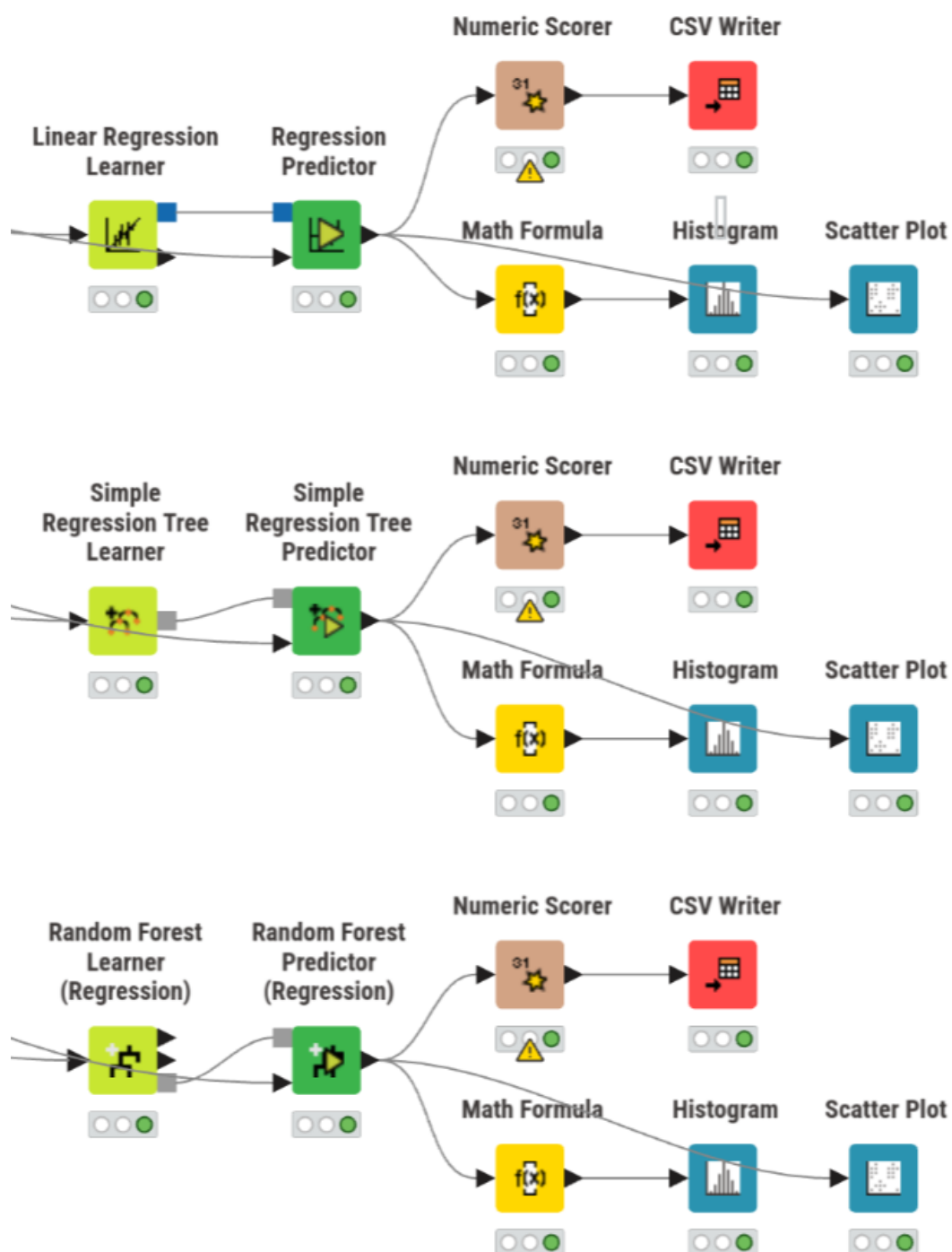


Figura 14: Workflow da fase de Regressão no KNIME

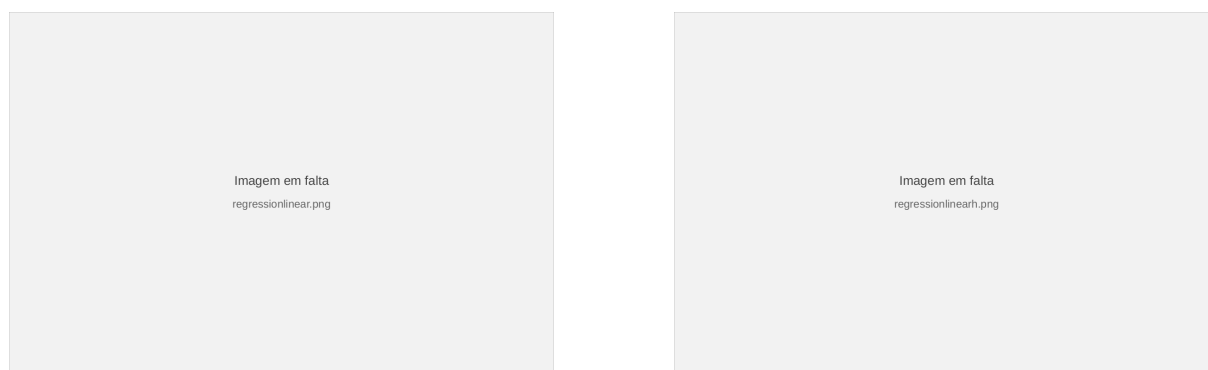


Figura 15: Diagnóstico do modelo linear: dispersão (esquerda) e histograma de resíduos (direita)

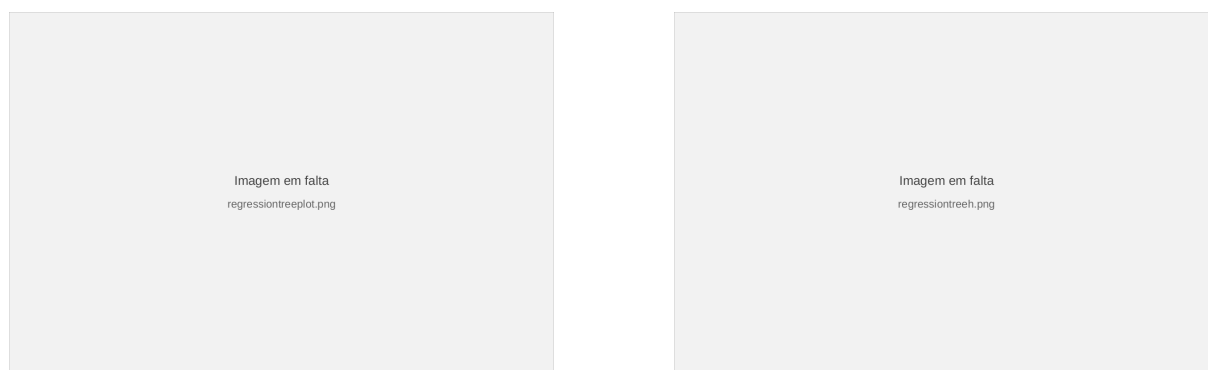


Figura 16: Diagnóstico da árvore de regressão: dispersão (esquerda) e histograma de resíduos (direita)

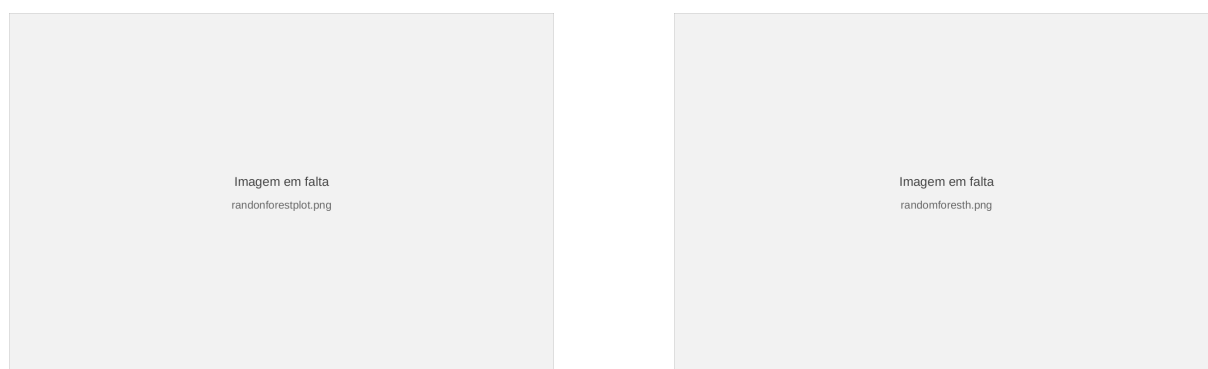


Figura 17: Diagnóstico da *Random Forest*: dispersão (esquerda) e histograma de resíduos (direita)

Parte II — Tarefa 2: Dataset Atribuído (Consumo Energético)

5 Definição da metodologia

A metodologia de análise de dados utilizada neste problema é a **SEMMA**: *Sample, Explore, Modify, Model e Assess*. A fase de *Sample* encontra-se já concretizada na forma do dataset atribuído, pelo que o presente relatório se foca nas fases seguintes. Assim, começou-se por realizar a fase de *Explore* (Exploração), cujo objetivo é perceber potenciais tendências e problemas com os dados, para na fase de *Modify* (Modificação) se fazerem transformações que visam corrigir os problemas identificados. Por fim, aplicam-se as fases de *Model* (Modelação) e *Assess* (Avaliação) para cada um dos 5 modelos desenvolvidos.

5.1 Domínio e dataset

O dataset atribuído contém registos sobre consumo energético em diferentes instalações, enriquecidos com variáveis meteorológicas (temperatura, humidade, precipitação, velocidade do vento, radiação solar, entre outras) e temporais. O objetivo é classificar o nível de consumo energético (**Consumptions**) em 5 categorias ordenadas: *Low*, *MediumLow*, *Medium*, *MediumHigh* e *High*.

O dataset apresenta cerca de **8 700 registos** no total, com múltiplas features numéricas e algumas categóricas, e foi distribuído com os problemas de qualidade que a metodologia SEMMA prevê tratar nas fases de exploração e modificação.

5.2 Visão geral do workflow desenvolvido no KNIME

De forma geral, o workflow está organizado num conjunto de metanodes que encapsulam os diferentes passos da metodologia SEMMA. O nó **String to Number**, que converte features numéricas que aparecem no dataset em formato string, encontra-se fora de qualquer metanode de forma a propagar os missing values nessas features para a Exploração e para a Modificação simultaneamente. No final, cada um dos 5 modelos recebe o output das transformações dos dados, sendo a Avaliação feita dentro de cada metanode.

6 Exploração

O primeiro passo da exploração dos dados foi a utilização do nó **Statistics** para verificar diversas estatísticas, destacando-se as seguintes observações:

- **Precipitation** e **rain** apresentam muitos missing values (~90%);
- **Snowfall** tem todos os valores a 0 (mean = min = max = 0);
- **RowID** é redundante;
- Todas as restantes features apresentam uma quantidade semelhante de missing values, cerca de 10%. Por isso, deixa de ser viável remover as entradas com missing values, uma vez que, feito isso, o dataset final fica com ≈1300 entradas (apenas 15% do total). Portanto,

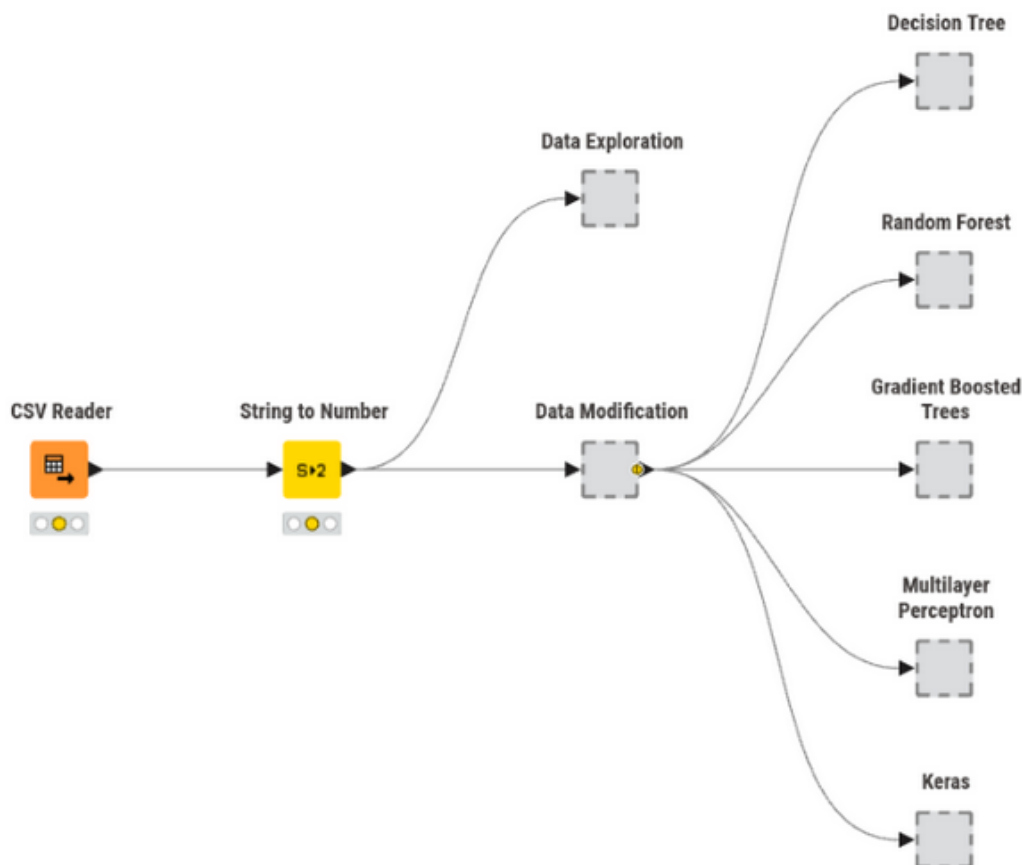


Figura 18: Visão geral do workflow desenvolvido no KNIME para o dataset atribuído

foram avaliadas, para cada modelo, diferentes abordagens para imputar os missing values (média e mediana para os numéricos e moda ou valor fixo “Missing” para as strings).

De seguida, foi verificada a presença de outliers, recorrendo tanto a **Numeric Outliers** para obter resultados numéricos como ao **Box Plot** para fazer uma exploração mais visual. Desta análise, retirou-se que várias features (como as do vento e as da voltagem) apresentam outliers em quantidade reduzida ($<2,5\%$) e a **surface_pressure** ligeiramente mais ($\sim 5\%$). No total, são 1700 as entradas com algum outlier, pelo que a sua remoção não foi considerada, pois perder-se-ia uma boa parte do dataset (cerca de 20%). Portanto, na avaliação dos modelos, os outliers foram ou mantidos ou ajustados para o valor permitido mais próximo, de maneira a avaliar qual o melhor tratamento para cada modelo.

Por fim, foi ainda feita uma análise dos valores das features categóricas com **Value Counter**, para determinar se existem valores fora do padrão. Concluiu-se que as features **Diffuse** e **Direct Radiation** não apresentam valores anormais. Já a target, **Consumptions**, apresenta, em algumas entradas, valores com erros ortográficos (e.g., *Hgh* e *Lw*) ou escritos em português em vez de inglês (e.g., *MedioAlto* vs. *Medium-High Consumption*).

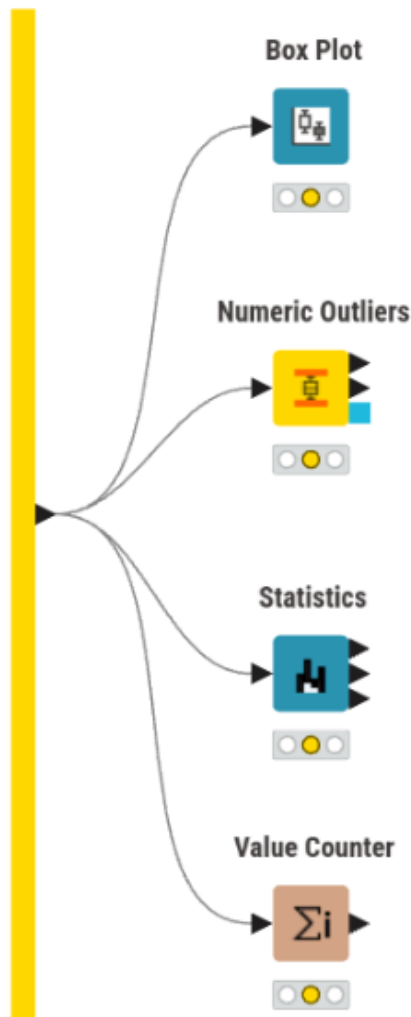


Figura 19: Metanode de Exploração no KNIME

7 Transformação

As primeiras transformações aplicadas aos dados centraram-se nos formatos das features, incluindo a conversão das colunas numéricas que se encontravam em string (**Medium Voltage** e **Total**), que foi realizada fora deste metanode, conforme descrito anteriormente, e a passagem das datas de string para um formato adequado, além da consequente extração dos campos mais relevantes para este problema (hora, dia da semana e mês). A feature original com a data completa foi removida para evitar introduzir ruído. Ainda, foram detectadas 4 linhas nas quais a data era inválida (e.g., 2025-11-32), pelo que essas entradas foram removidas, por ser uma quantidade muito reduzida.

De seguida, no seguimento da exploração realizada, foram removidas algumas features:

- **RowID** (redundante);
- **precipitation** e **rain** (~90% missing values);
- **snowfall** (redundante, mean = min = max = 0).

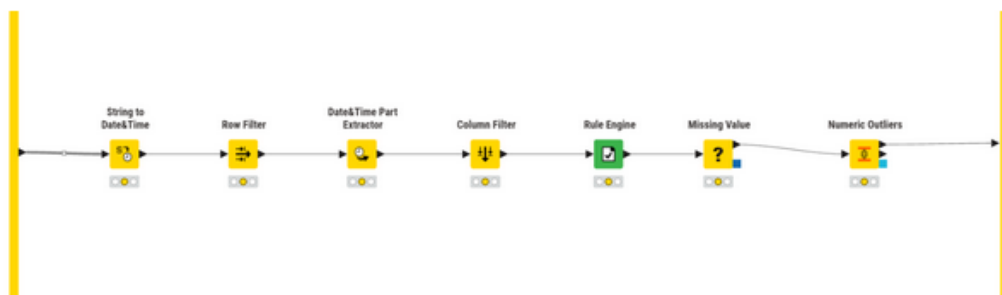


Figura 20: Metanode de Transformação no KNIME

Por fim, de acordo com o que foi possível perceber na exploração, foi aplicada uma **Rule Engine** para uniformizar o formato dos valores de **Consumptions**, resultando daqui 5 valores possíveis: *Low*, *MediumLow*, *Medium*, *MediumHigh* e *High*.

8 Modelação e Avaliação

É agora altura de desenvolver modelos para o presente problema. Foram concebidos 5 modelos de classificação, com 3 baseados em árvores e 2 redes neurais. Algumas observações relevantes para as secções seguintes:

1. A partição dos dados para treino e teste foi feita, sempre que possível, com recurso aos nós **X-Partitioner** e **X-Aggregator**, uma vez que o uso de cross-validation, embora mais custoso a nível computacional, permite aproveitar a totalidade do dataset e reduzir as chances do modelo sofrer de overfitting;
2. Os nodos de cross-validation foram configurados com uma random seed, de modo a ser possível obter resultados reproduzíveis e sem variabilidade aleatória;
3. Não foram testadas todas as combinações possíveis de transformações de features e configurações dos modelos, pois é uma quantidade impraticável. Por isso, a abordagem adotada foi: testar um conjunto de abordagens e as suas combinações; tirar daí o melhor resultado; realizar o próximo conjunto de testes com a combinação obtida anteriormente. Isto pode fazer com que não sejam detectadas as melhores combinações para cada modelo, mas torna este processo de refinamento mais exequível.

8.1 Decision Tree

Tal como mencionado na fase de exploração, os tratamentos de missing values testados foram média e mediana para os valores numéricos e, para os categóricos, moda e valor fixo “Missing”. Já os outliers, tal como dito anteriormente, foram mantidos ou ajustados para o valor permitido mais próximo.

Os resultados obtidos mostram que o tratamento de missing values e outliers teve impacto reduzido no desempenho da Decision Tree, verificando-se diferenças muito pequenas entre as diferentes estratégias avaliadas (menos de 1%). Em contrapartida, os hiperparâmetros estruturais da árvore tiveram mais influência no desempenho final. Em particular, o aumento do número de records por nó levou a melhorias consistentes da accuracy e do Cohen’s Kappa. Observou-se

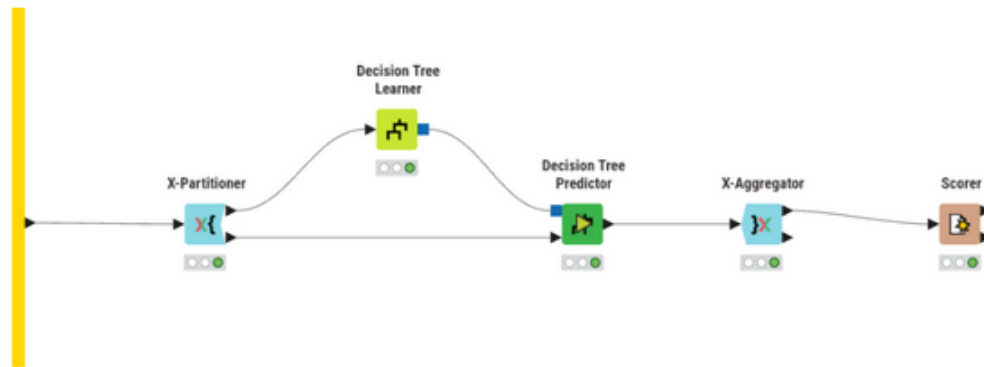


Figura 21: Metanódo do modelo Decision Tree no KNIME

Tabela 8: Decision Tree — impacto das estratégias de pré-processamento

Outliers	Missing (números)	Missing (categóricos)	Accuracy	Cohen's Kappa
manter	média	moda	0.884	0.852
manter	média	fixo	0.887	0.855
manter	mediana	moda	0.885	0.853
manter	mediana	fixo	0.888	0.857
ajustar	média	moda	0.885	0.852
ajustar	média	fixo	0.890	0.859
ajustar	mediana	moda	0.885	0.853
ajustar	mediana	fixo	0.889	0.858

também que a utilização de pruning não trouxe vantagens relevantes relativamente à ausência de pruning, obtendo resultados ligeiramente inferiores na maioria das configurações, mas reduzindo a variabilidade ao ajustar os outros parâmetros. Quanto à métrica de divisão utilizada, tanto o Gini Index como o Gain Ratio produziram desempenhos semelhantes, embora o Gini Index tenha apresentado, de forma consistente, resultados ligeiramente superiores. O **melhor desempenho** global foi obtido com Gini Index, sem pruning e com mínimo de 12 registos por nó, atingindo uma accuracy de **0.927** e um Cohen's Kappa de **0.906**.

8.2 Random Forest

Aqui foram testadas as mesmas combinações de tratamento de missing values e outliers que na Decision Tree.

Tal como observado na Decision Tree, as diferentes estratégias de tratamento de missing values e outliers tiveram impacto praticamente nulo no desempenho da Random Forest. Relativamente aos hiperparâmetros, verificou-se que limitar o número de níveis das árvores provocou uma redução consistente do desempenho. Isto sugere que este problema beneficia de árvores mais profundas, capazes de capturar relações mais complexas entre as features. Também o aumento do parâmetro *Minimum child node size* levou a pequenas perdas de desempenho. Os diferentes critérios de divisão testados apresentaram desempenhos muito semelhantes, embora o Information Gain Ratio tenha obtido, de forma consistente, os melhores resultados. Finalmente,

Tabela 9: Decision Tree — variação dos hiperparâmetros estruturais

Quality Measure	Pruning Method	Min recs por nó	Accuracy	Cohen's Kappa
Gini index	No pruning	2	0.890	0.859
Gini index	No pruning	4	0.897	0.869
Gini index	No pruning	8	0.919	0.897
Gini index	No pruning	12	0.927	0.906
Gini index	MDL	2	0.925	0.904
Gini index	MDL	4	0.925	0.904
Gini index	MDL	8	0.925	0.904
Gini index	MDL	12	0.926	0.905
Gain ratio	No pruning	2	0.888	0.857
Gain ratio	No pruning	4	0.904	0.878
Gain ratio	No pruning	8	0.918	0.895
Gain ratio	No pruning	12	0.922	0.901
Gain ratio	MDL	2	0.919	0.897
Gain ratio	MDL	4	0.919	0.897
Gain ratio	MDL	8	0.921	0.899
Gain ratio	MDL	12	0.921	0.899

a avaliação do número de modelos mostrou que aumentar o número de árvores acima de 100 não produziu melhorias relevantes, indicando que a Random Forest atinge rapidamente estabilidade neste problema. O **melhor desempenho** foi obtido com Information Gain Ratio, sem limitar profundidade, sem minimum child node size e com 100 modelos, atingindo uma accuracy de **0.930** e Cohen's Kappa de **0.910**.

8.3 Gradient Boosted Trees

Ao contrário do que foi feito até agora, não foram avaliadas as estratégias de imputação de missing values, uma vez que este modelo dispõe de mecanismos internos para lidar com este problema. Sendo assim, foram testados estes métodos juntamente com o tratamento de outliers com as configurações padrão, antes de começar a variar os hiperparâmetros.

É possível verificar que a configuração com ajuste de outliers tem um resultado igual a não tratar outliers (usando em ambos os casos XGBoost), pelo que se optou por utilizar a abordagem sem ajuste, por não introduzir dados artificiais. Na avaliação que se segue, a variação do learning rate foi feita em conjunto com o número de modelos, ou seja, diminuindo a learning rate, aumentasse o número de modelos. Isto foi feito porque, ao diminuir a learning rate, cada árvore individual contribui menos para o resultado final, pelo que é necessário aumentar o total de árvores para compensar.

Os resultados mostram que diferentes combinações de learning rate e número de modelos conduzem praticamente ao mesmo desempenho máximo (accuracy = **0.932** e Cohen's Kappa = **0.913**), sugerindo que, para este problema, o aumento do número de modelos juntamente com a redução da learning rate não produzem ganhos relevantes. Em contrapartida, observou-se que configurações com maior profundidade das árvores (limit levels = 8) e com row sampling ativo tendem a apresentar desempenhos ligeiramente superiores, indicando que estes parâmetros têm

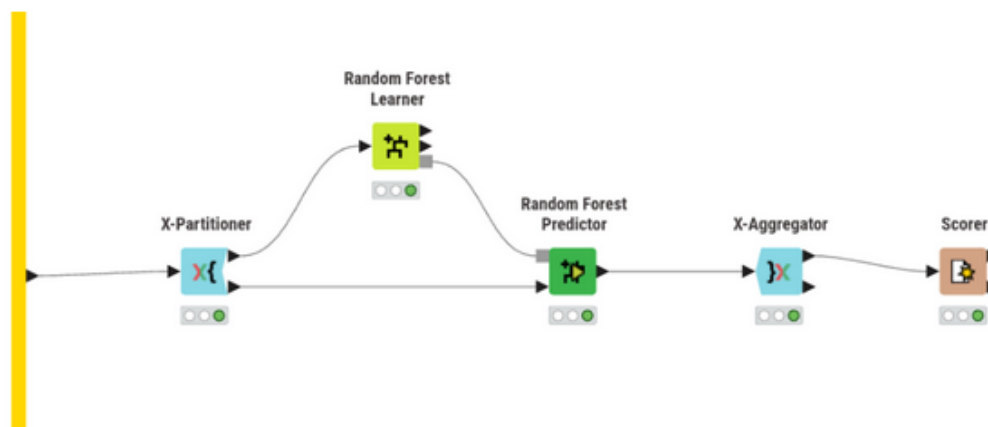


Figura 22: Metanódo do modelo Random Forest no KNIME

Tabela 10: Random Forest — impacto das estratégias de pré-processamento (100 modelos)

Outliers	Missing (números)	Missing (categóricos)	Accuracy	Cohen's Kappa
manter	média	moda	0.929	0.909
manter	média	fixo	0.929	0.909
manter	mediana	moda	0.930	0.910
manter	mediana	fixo	0.929	0.909
ajustar	média	moda	0.929	0.909
ajustar	média	fixo	0.928	0.908
ajustar	mediana	moda	0.929	0.909
ajustar	mediana	fixo	0.928	0.908

maior influência no resultado final do que a combinação learning rate / número de modelos.

8.4 Multilayer Perceptron

Para as redes neurais, uma transformação de dados importante é a normalização (e consequente desnormalização), pelo que foram testados 3 métodos diferentes para a normalização dos dados. No caso do método Min-max, usar o intervalo $[0; 1]$ produziu resultados bastante medíocres. Por isso, o intervalo usado foi $[-1; 1]$, que melhorou a performance da rede por centrar os dados em zero. Ainda, o método decimal scaling também produziu resultados pouco satisfatórios (accuracy na ordem dos 60%), pelo que não foi avaliado com o mesmo detalhe que foram os outros.

A outra transformação essencial tem a ver com as features categóricas nominais, que não são adequadas para dar como input a uma rede neuronal. Foram testadas duas formas de as tornar apropriadas ao modelo:

- **numerar:** atribuir valores numéricos sequenciais às categorias. Esta forma de tratamento pode não ser 100% adequada porque introduz na feature uma noção de ordem que pode não ser verdadeira/aplicável. No entanto, para este problema, as features afetadas usam classificações como *None*, *Low*, *Medium* e *High*, pelo que a numeração pode fazer sentido;
- **One to Many:** nó que, para cada categoria de uma feature, cria uma nova feature cujo

Tabela 11: Random Forest — variação dos hiperparâmetros (100 modelos, outliers: manter, missing: mediana/moda)

Split criterion	Limit levels	Min child node	Accuracy	Cohen's Kappa
Information Gain Ratio	off	off	0.930	0.910
Information Gain Ratio	off	2	0.929	0.909
Information Gain Ratio	off	4	0.927	0.907
Information Gain Ratio	10	off	0.919	0.896
Information Gain Ratio	10	2	0.919	0.896
Information Gain Ratio	10	4	0.918	0.895
Information Gain	off	off	0.929	0.909
Information Gain	off	2	0.928	0.908
Information Gain	off	4	0.927	0.907
Information Gain	10	off	0.922	0.900
Information Gain	10	2	0.921	0.899
Information Gain	10	4	0.921	0.898
Gini	off	off	0.929	0.909
Gini	off	2	0.928	0.907
Gini	off	4	0.927	0.907
Gini	10	off	0.920	0.898
Gini	10	2	0.920	0.897
Gini	10	4	0.921	0.899

nome é o nome da original junto com cada valor, e atribui 0 ou 1 conforme o valor da feature naquela entrada. Este método permite tornar features categóricas adequadas para redes neurais, sem introduzir noções de ordem como a simples numeração.

Um detalhe importante, que difere consoante qual das duas abordagens é utilizada, é quando se faz a normalização. No caso da numeração, a normalização é feita depois, de modo a englobar os novos valores criados. Assim, evita-se que a feature fique com uma escala diferente das restantes. Já no caso de se usar o One to Many, a normalização é feita antes, pois os valores já estão numa escala consistente (0 ou 1) e assim não se destrói a interpretação binária, que é precisamente o objetivo desta abordagem.

Considerando tudo isto, começou-se por avaliar todas as combinações destas configurações. O único método de normalização não avaliado com a mesma profundidade que os outros dois foi o Decimal Scaling, porque fez o modelo obter uma accuracy de ~60% com algumas combinações variadas dos restantes parâmetros em avaliação. Assim, foi possível reduzir ligeiramente o espaço de procura (48 para 32 combinações).

A melhor combinação encontrada foi: normalização Z-score, One to Many para as variáveis categóricas, ajuste de outliers, mediana para missing values numéricos e moda para missing values categóricos. Com esta combinação, procedeu-se à avaliação dos hiperparâmetros da rede.

Os resultados obtidos mostram que o desempenho do Multilayer Perceptron depende mais das transformações aplicadas aos dados do que os modelos anteriores, com ênfase para a normalização e a representação das variáveis categóricas. Dentre os métodos testados, a normalização Z-score apresentou consistentemente melhores resultados do que Min-max e Decimal Scaling.

Tabela 12: Random Forest — variação do número de modelos

Number of models	Accuracy	Cohen's Kappa
50	0.929	0.909
100	0.930	0.910
150	0.930	0.910
200	0.929	0.909
250	0.929	0.909

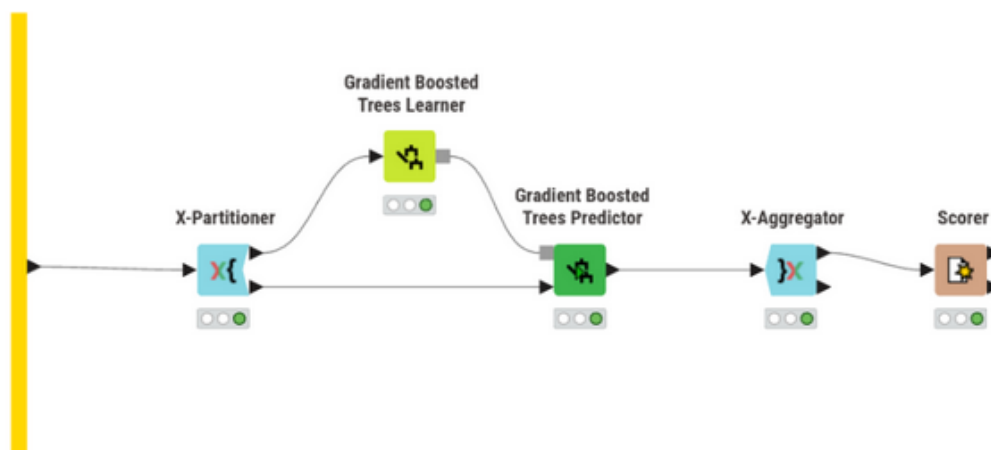


Figura 23: Metanode do modelo Gradient Boosted Trees no KNIME

Relativamente às variáveis categóricas, a abordagem One to Many revelou-se superior à simples numeração das categorias neste problema.

Quanto à arquitetura da rede, verificou-se que aumentar o número de iterações melhorou consistentemente os resultados, indicando que o modelo beneficiou de mais tempo de treino. No entanto, o aumento do número de hidden layers e neurónios nem sempre conduziu a melhorias. Neste problema, os resultados sugerem que arquiteturas relativamente simples já conseguem capturar os padrões mais relevantes presentes no dataset. Assim, o aumento da profundidade e do número de neurónios introduziu maior complexidade de otimização sem acrescentar capacidade útil de generalização em relação aos dados de treino, levando em vários casos a uma degradação do desempenho no conjunto de teste. O **melhor resultado** foi obtido com 300 iterações, 1 hidden layer e 10 neurónios, atingindo uma accuracy de **0.903** e Cohen's Kappa de **0.875**.

8.5 Keras

O último modelo avaliado foi mais uma rede neuronal, desta vez usando a biblioteca Keras, que possibilita explorar arquiteturas mais flexíveis e diferentes métodos de otimização. Portanto, as transformações dos dados avaliadas para o MLP não foram reavaliadas, tendo-se optado por manter a melhor combinação encontrada (normalização Z-score, One to Many para as features categóricas, ajuste de outliers, mediana para os missing values numéricos e moda para os missing values categóricos). Portanto, agora o foco passa a ser a definição da arquitetura da rede e do processo de treino.

Antes de prosseguir para a arquitetura da rede, é necessário efetuar algumas transformações

Tabela 13: Gradient Boosted Trees — avaliação dos métodos de missing values e outliers

Outliers	Missing values	Accuracy	Cohen's Kappa
manter	XGBoost	0.928	0.908
manter	Surrogate	0.897	0.868
ajustar	XGBoost	0.928	0.908
ajustar	Surrogate	0.896	0.867

Tabela 14: Gradient Boosted Trees — variação dos hiperparâmetros

Learning Rate	Nº Models	Limit levels	Row Sampling	Accuracy	Kappa
0.1	100	4	off	0.928	0.908
0.1	100	4	0.5	0.929	0.909
0.1	100	4	0.8	0.929	0.909
0.1	100	8	off	0.929	0.909
0.1	100	8	0.5	0.931	0.912
0.1	100	8	0.8	0.932	0.913
0.05	200	4	off	0.928	0.908
0.05	200	4	0.5	0.929	0.909
0.05	200	4	0.8	0.929	0.909
0.05	200	8	off	0.928	0.908
0.05	200	8	0.5	0.932	0.913
0.05	200	8	0.8	0.932	0.912
0.02	400	4	off	0.928	0.908
0.02	400	4	0.5	0.928	0.908
0.02	400	4	0.8	0.928	0.908
0.02	400	8	off	0.930	0.910
0.02	400	8	0.5	0.932	0.913
0.02	400	8	0.8	0.932	0.913

nos dados para ser possível aplicar a rede ao problema de classificação. Em particular, é preciso transformar a target **Consumptions**, pois ao contrário do RProp MLP, estas redes neuronais só recebem e devolvem valores numéricos como resultado. Desta forma, foram testadas duas formas de fazer esta transformação:

- **One to Many:** cria-se uma coluna para cada possível valor da feature original, ou seja, cria-se um conjunto de booleanos. Para o output da rede, usam-se 5 neurónios, obtendo-se 5 valores de 0 a 1 que somam 1, ou seja, probabilidades. Depois é só escolher o maior para se fazer a previsão daquela entrada. Embora este formato de 0s ou 1s já seja aceite pela rede, a noção lógica destes valores é perdida, pelo que esta poderá não ser a melhor abordagem;
- **numerar:** numerar de 1 a 5 os possíveis valores de **Consumptions**. Este tratamento é adequado porque os valores da target já têm uma ordem própria (Low < MediumLow < Medium < MediumHigh < High), pelo que o significado da numeração continua a fazer sentido para a rede. Aqui, a camada de output passa a ter apenas 1 neurónio. O valor

Tabela 15: MLP — impacto da normalização, encoding e pré-processamento (100 iter., 1 hidden layer, 10 neurónios)

Normalização	Categ.	Outliers	Miss. (núm.)	Miss. (cat.)	Acc.	Kappa
Min-max	numerar	manter	média	moda	0.839	0.793
Min-max	numerar	manter	média	fixo	0.845	0.802
Min-max	numerar	manter	mediana	moda	0.841	0.796
Min-max	numerar	manter	mediana	fixo	0.840	0.795
Min-max	numerar	ajustar	média	moda	0.847	0.803
Min-max	numerar	ajustar	média	fixo	0.838	0.792
Min-max	numerar	ajustar	mediana	moda	0.858	0.818
Min-max	numerar	ajustar	mediana	fixo	0.864	0.826
Min-max	One to Many	manter	média	moda	0.855	0.814
Min-max	One to Many	manter	média	fixo	0.848	0.805
Min-max	One to Many	manter	mediana	moda	0.857	0.817
Min-max	One to Many	manter	mediana	fixo	0.841	0.796
Min-max	One to Many	ajustar	média	moda	0.857	0.817
Min-max	One to Many	ajustar	média	fixo	0.856	0.816
Min-max	One to Many	ajustar	mediana	moda	0.860	0.821
Min-max	One to Many	ajustar	mediana	fixo	0.853	0.812
Z-score	numerar	manter	média	moda	0.862	0.823
Z-score	numerar	manter	média	fixo	0.862	0.823
Z-score	numerar	manter	mediana	moda	0.860	0.820
Z-score	numerar	manter	mediana	fixo	0.860	0.821
Z-score	numerar	ajustar	média	moda	0.864	0.825
Z-score	numerar	ajustar	média	fixo	0.858	0.817
Z-score	numerar	ajustar	mediana	moda	0.867	0.830
Z-score	numerar	ajustar	mediana	fixo	0.856	0.815
Z-score	One to Many	manter	média	moda	0.882	0.849
Z-score	One to Many	manter	média	fixo	0.857	0.817
Z-score	One to Many	manter	mediana	moda	0.876	0.841
Z-score	One to Many	manter	mediana	fixo	0.862	0.823
Z-score	One to Many	ajustar	média	moda	0.878	0.844
Z-score	One to Many	ajustar	média	fixo	0.858	0.819
Z-score	One to Many	ajustar	mediana	moda	0.886	0.854
Z-score	One to Many	ajustar	mediana	fixo	0.858	0.818

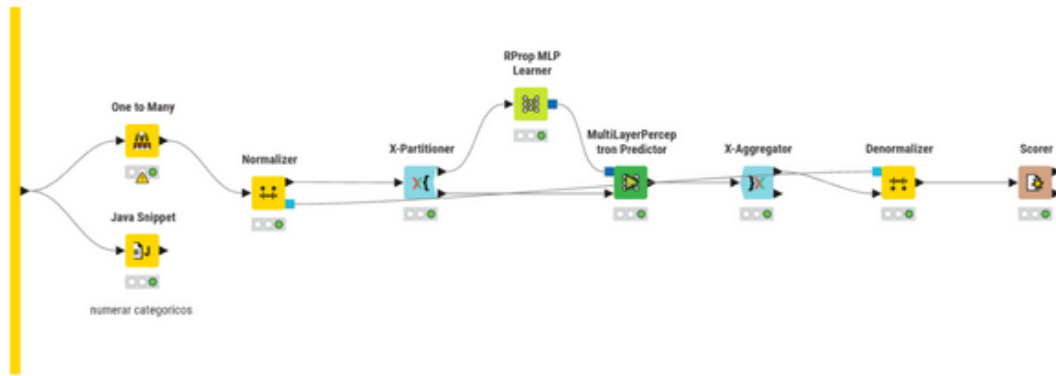


Figura 24: Metanode do modelo Multilayer Perceptron no KNIME

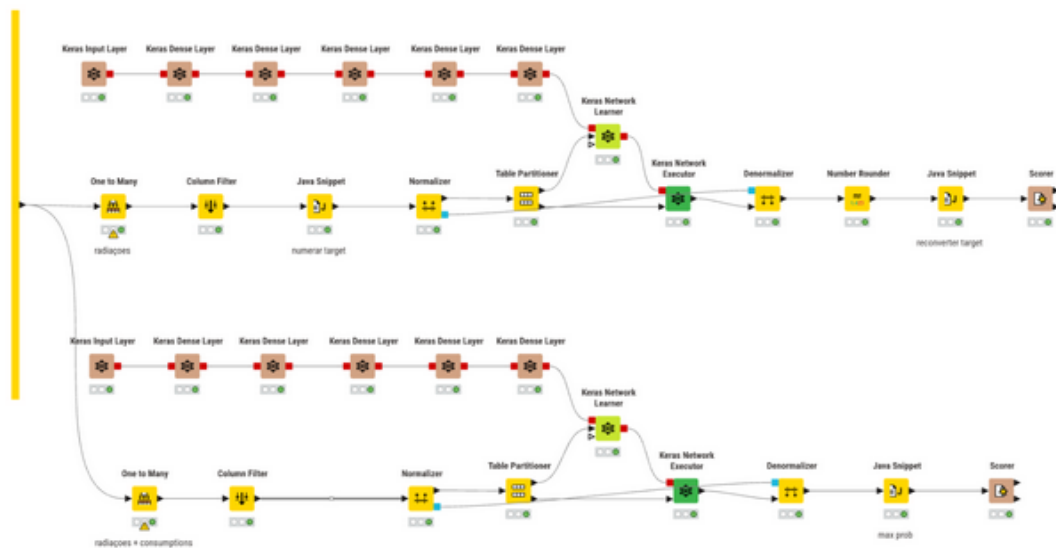


Figura 25: Metanode do modelo Keras no KNIME

gerado pela rede é limitado ao intervalo válido e depois arredondado (e.g., $6.1 \Rightarrow 5$ e $0.2 \Rightarrow 1$), sendo por fim feita a conversão inversa para o valor em string correspondente.

Como configuração inicial, usou-se uma hidden layer com 16 neurónios e função de ativação ReLU (tal como em todas as futuras hidden layers adicionadas). No caso do One to Many, a loss function usada é a Categorical Cross Entropy com função de ativação Softmax. No caso da numeração, a loss function é a Mean Absolute Error com função de ativação Linear. O otimizador utilizado em ambas as situações foi o Adam, com os parâmetros padrão. O uso de cross-validation tornou-se computacionalmente inviável com este modelo, pelo que foi usado um Table Partitioner, fazendo uma partição de 70/30 com uma random seed fixa.

Os resultados obtidos mostram que a estratégia One to Many é significativamente mais adequada para este problema de classificação multiclasse, atingindo desempenhos superiores de forma consistente em praticamente todas as arquiteturas testadas. Além disso, esta abordagem demonstrou maior estabilidade, apresentando melhorias mais graduais à medida que se aumentava a complexidade da rede.

Tabela 16: MLP — variação dos hiperparâmetros de arquitetura (melhor pré-processamento)

Nº iterations	Nº hidden layers	Nº neurons/layer	Accuracy	Cohen's Kappa
100	1	10	0.886	0.854
100	1	20	0.885	0.853
100	1	30	0.882	0.848
100	1	40	0.885	0.853
100	2	10	0.843	0.798
100	2	20	0.870	0.833
100	2	30	0.879	0.845
100	2	40	0.867	0.830
100	3	10	0.829	0.780
100	3	20	0.830	0.782
100	3	30	0.841	0.797
100	3	40	0.856	0.816
200	1	10	0.900	0.873
200	1	20	0.899	0.871
200	1	30	0.896	0.867
200	1	40	0.893	0.863
200	2	10	0.866	0.828
200	2	20	0.889	0.859
200	2	30	0.879	0.845
200	2	40	0.869	0.833
300	1	10	0.903	0.875
300	1	20	0.900	0.872
300	1	30	0.895	0.865
300	1	40	0.888	0.856
300	2	10	0.884	0.851
300	2	20	0.889	0.858
300	2	30	0.877	0.843
300	2	40	0.865	0.828

Por outro lado, a abordagem baseada na numeração da target, embora inicialmente apresente resultados bastante inferiores, revelou uma maior sensibilidade ao aumento da capacidade da rede e do número de epochs, registando melhorias mais acentuadas com arquiteturas mais profundas e mais tempo de treino. Este comportamento pode ser explicado pelo facto de esta estratégia exigir da rede uma aprendizagem mais precisa das relações entre os diferentes níveis de consumo — por exemplo, uma determinada entrada pode ter Consumption MediumHigh (4) e a rede dar como output 3.4, que será arredondado para 3 (Medium), embora esteja muito perto do resultado correto.

O **melhor resultado** global do Keras foi obtido com a estratégia One to Many, arquitetura de 4 hidden layers (128, 64, 32, 16 neurónios) e 20 epochs, atingindo uma accuracy de **0.842** e Cohen's Kappa de **0.798**. Ainda assim, este resultado fica consideravelmente abaixo dos melhores modelos baseados em árvores, o que é esperado dado o menor número de dados de

Tabela 17: Keras — One to Many (target): variação da arquitetura (5 epochs)

Nº hidden layers	Nº units	Accuracy	Cohen's Kappa
1	16	0.576	0.457
2	16, 8	0.639	0.537
2	32, 16	0.724	0.647
3	32, 16, 8	0.752	0.682
3	64, 32, 16	0.777	0.715
4	128, 64, 32, 16	0.779	0.718

Tabela 18: Keras — Numerar (target): variação da arquitetura (5 epochs)

Nº hidden layers	Nº units	Accuracy	Cohen's Kappa
1	16	0.308	0.116
2	16, 8	0.297	0.102
2	32, 16	0.377	0.201
3	32, 16, 8	0.360	0.182
3	64, 32, 16	0.434	0.272
4	128, 64, 32, 16	0.513	0.371

treino disponível para redes neurais (partição 70/30 sem cross-validation).

9 Conclusão

9.1 Síntese dos resultados do dataset atribuído

Seguindo a metodologia SEMMA, começou-se por explorar o dataset de forma a identificar problemas de qualidade dos dados, padrões relevantes e possíveis transformações necessárias para a modelação. A análise realizada permitiu detectar valores em falta, outliers, inconsistências na target e features redundantes, conduzindo posteriormente à aplicação de diferentes estratégias de pré-processamento e transformação dos dados para resolver esses problemas.

Após a fase de modificação, foram desenvolvidos e avaliados cinco modelos de classificação distintos. A Tabela 20 sintetiza os melhores resultados obtidos para cada modelo.

De forma geral, os resultados mostram que os modelos baseados em árvores apresentaram o melhor desempenho neste problema, sobretudo os métodos de ensemble (Random Forest e Gradient Boosted Trees). Estes modelos revelaram também maior robustez relativamente às transformações aplicadas aos dados, sendo pouco sensíveis às diferentes estratégias de tratamento de missing values e outliers.

Por outro lado, as redes neurais demonstraram maior dependência da preparação dos dados. Em particular, a escolha do método de normalização e da representação das variáveis categóricas teve impacto significativo no desempenho obtido. O modelo desenvolvido com Keras destacou-se pela maior flexibilidade e capacidade de adaptação da arquitetura da rede, permitindo explorar diferentes formas de representação da target e diferentes configurações de treino. Ainda assim, os resultados obtidos permaneceram inferiores aos melhores modelos baseados em árvores, principalmente devido à impossibilidade de usar cross-validation por limitações computacionais, o

Tabela 19: Keras — variação do número de epochs (melhor arquitetura por estratégia)

Estratégia	Nº epochs	Accuracy	Cohen's Kappa
One to Many	5	0.779	0.718
One to Many	10	0.803	0.747
One to Many	20	0.842	0.798
Numerar	5	0.513	0.371
Numerar	10	0.639	0.533
Numerar	20	0.684	0.592

Tabela 20: Comparação dos melhores resultados por modelo — Dataset Atribuído

Modelo	Melhor Accuracy	Melhor Cohen's Kappa
Decision Tree	0.927	0.906
Random Forest	0.930	0.910
Gradient Boosted Trees	0.932	0.913
Multilayer Perceptron (RProp)	0.903	0.875
Keras	0.842	0.798

que reduziu o volume de dados de treino disponível.

Em termos globais, o problema do dataset atribuído revelou-se adequado para modelos de classificação baseados em árvores, sendo o Gradient Boosted Trees o modelo com melhor desempenho global. Este resultado é consistente com a natureza do problema: as relações entre as variáveis meteorológicas e o consumo energético envolvem padrões temporais e não-lineares que beneficiam de modelos com elevada capacidade de captura de interações entre features.

9.2 Síntese global do trabalho

O trabalho desenvolvido abrangeu duas tarefas complementares que permitiram explorar diferentes dimensões da aprendizagem automática. A Tarefa 1 demonstrou que features acústicas do Spotify contêm sinal discriminativo genuíno — suficiente para classificar géneros musicais com cerca de 68% de exatidão e para identificar 5 perfis acústicos distintos através de *k-means* — mas insuficiente para prever a popularidade com qualidade razoável ($R^2 < 0.05$). A Tarefa 2 confirmou que os modelos de ensemble baseados em árvores são altamente eficazes para problemas de classificação com features heterogêneas e missing values, atingindo accuracies superiores a 93% no melhor caso.

O uso da plataforma KNIME permitiu uma implementação modular e reproduzível de ambas as tarefas, com workflows claramente estruturados que facilitaram a experimentação sistemática de hiperparâmetros e estratégias de pré-processamento.